



**BHARATI VIDYAPEETH'S
INSTITUTE OF COMPUTER APPLICATIONS & MANAGEMENT**

(Affiliated to Guru Gobind Singh Indraprastha
University, Approved by AICTE, New Delhi)

**Software Testing Lab
(MCA-168)
Practical File**

Submitted To:

Dr. Sania Kukkar

(Assistant Professor, BVICAM, New Delhi)

Submitted By:

Piyush Goel
{09311604425}

MCA 2nd SEM ,SECB

INDEX

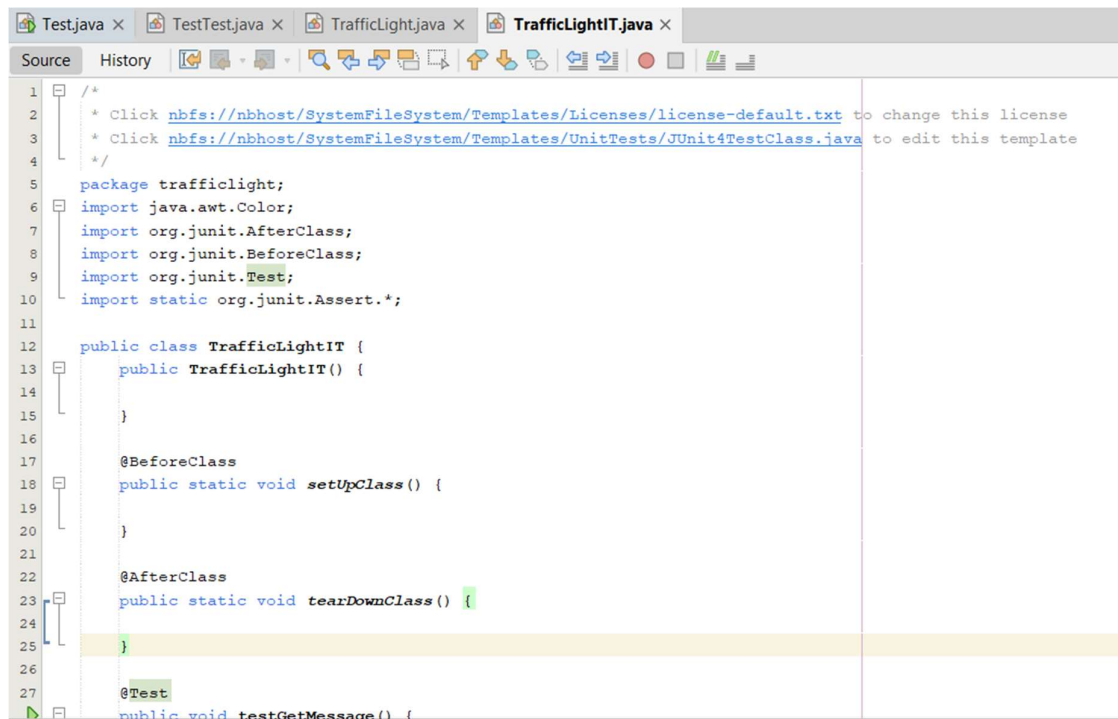
S.No.	Problem Description	Date	Sign.
P1	Construct proper test cases for traffic light system using radio buttons. The application should provide three radio buttons labeled Red, Yellow, and Green. When user selects Red: display "STOP" in red color; Yellow: display "READY" in yellow color; Green: display "GO" in green color. Implement traffic light logic in a separate unit-testable class. Write JUnit test cases to verify correctness of traffic light messages.		
P2	Demonstrate test-driven development through a java file, Overloading.java, which properly tests the Overloading.java class to ensure that Method Overloading modules are working properly.		
P3	To study and implement unit testing for a Java program that performs addition of two numbers using JUnit framework.		
P4	To design and test a Java program that checks whether a given number is even or odd using JUnit assertion methods.		
P5	Implement and test a Java program that verifies whether a given string is a palindrome using JUnit test cases and assertions.		
P6	Demonstrate test-driven development through a Java program to find the maximum of two numbers and test the logic using JUnit framework.		
P7	To create, demonstrate, and implement a Java program to calculate the factorial of a number and verify correctness using JUnit testing.		
P8	Demonstrate test-driven development through a simple project called Reverse in NetBeans. Reverse a string and test the result using JUnit unit testing methods.		
P9	Create and implement a Java program to calculate simple interest and verify the computation using JUnit test cases.		
P10	Construct proper test cases for a Program to validate user login credentials and test different input scenarios using JUnit testing.		
P11	Construct proper test cases for a general registration form. The test scenarios: i. Verify registration page loads successfully. ii. Verify required/mandatory fields are marked with * against the field. iii. Verify page has both submit and cancel/reset buttons. iv. Verify clicking submit after entering required fields submits data to server. v. Verify clicking cancel/reset cancels the request and resets all fields. vi. Verify not filling mandatory fields and clicking submit leads to a validation error.		
P12	In the class Arithmetic.java in Q3, add another sum() method. Also generate the corresponding sumTest() method through JUnit. Execute the Arithmetic Test class for inputs in which test cases fail as well as inputs where test cases pass. Record both test cases in proper test case templates.		
P13	Demonstrate test driven development through a project ProjectRepository. The project should contain a class		

	ProductRepository with appropriate methods for CRUD operations on a ProductRepository instance. Through JUnit create corresponding Test classes for each class. Aggregate these different tests in a test suite and execute the same.		
P14	Demonstrate test driven development through a project Addition. The project should contain a class Addition with appropriate methods for returning the sum of two numbers. Through JUnit create corresponding Parameterized Test classes and TestRunner classes to execute the above test on a collection of test cases. The test cases should be chosen using boundary value analysis technique for an input range from [1-10] for the first number and [11-20] for the second number.		
P15	Demonstrate test driven development through a project MessageUtility. The project should contain a class MessageUtil with appropriate methods for printing a Message. Through JUnit create corresponding Test classes and TestRunner classes to execute and test the above class.		
P16	Create a new project called AssertClass. Refactor the test class and TestRunner class of Question 15 to demonstrate the assertion methods of the assert class to write useful tests.		
P17	Create optimal parametrized tests for the login web page of a company website say XYZ Corporation. The Login feature is governed by the following specifications: A user can enter a valid email or invalid email. The user's email is in the database. A user enters a correct or incorrect password (a password that doesn't fit the account or password is invalid). A user can be blocked.		
P18	Create a trivial class Counter with the following: The constructor will create a counter and set it to 0. The increment method will add one to the counter and return the new value. The decrement method will subtract one from the counter and return the new value. Write test methods to test these methods. Also create a TestRunner class to execute these methods.		
P19	Demonstrate test driven development through a project ProjectRepository. The project should contain based on the following rules: a) Marks in Mathematics $\geq$ 60 b) Marks in Physics $\geq$ 50 c) Marks in Chemistry $\geq$ 50 d) Total in all three subjects $\geq$ 160 or e) Total in Maths & Physics $\geq$ 120. If aggregate of an eligible candidate is greater than 225, he will be eligible for honors course. Write appropriate class to represent this situation. Through JUnit create corresponding Test classes for each class. Aggregate these different tests in a test suite and execute the same.		
P20	Demonstrate test driven development through a class MyArray. Write a linear search method to find out the largest element of the array. Design a test class to test this method, using JUnit framework. Record both test cases in proper test case templates.		

Q1. Construct proper test cases for traffic light system using radio buttons. The application should provide three radio buttons labelled Red, Yellow, and Green.

- When the user selects:
 - Red, display the message “STOP” in red color.
 - Yellow, display the message “READY” in yellow color.
 - Green, display the message “GO” in green color.
- The message should appear above the radio buttons.
- Initially, no message should be displayed.
- Implement the traffic light logic in a separate class to make it unit testable.
- Write JUnit test cases to verify the correctness of the traffic light messages.

Answer:



```
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/UnitTests/JUnit4TestClass.java to edit this template
4   */
5   package trafficlight;
6   import java.awt.Color;
7   import org.junit.AfterClass;
8   import org.junit.BeforeClass;
9   import org.junit.Test;
10  import static org.junit.Assert.*;
11
12  public class TrafficLightIT {
13      public TrafficLightIT() {
14      }
15
16
17      @BeforeClass
18      public static void setUpClass() {
19      }
20
21
22      @AfterClass
23      public static void tearDownClass() {
24      }
25  }
26
27      @Test
28      public void testGetMessage() {
```

```
Test.java x TestTest.java x TrafficLight.java x TrafficLightIT.java x
22 @AfterClass
23 public static void tearDownClass() {
24
25 }
26
27 @Test
28 public void testGetMessage() {
29     System.out.println("getMessage");
30     String color = "red";
31     TrafficLight instance = new TrafficLight();
32     String expectedResult = "STOP";
33     String result = instance.getMessage(color);
34     assertEquals(expected: expectedResult, actual: result);
35 }
36
37 @Test
38 public void testGetDisplayColor() {
39     System.out.println("getDisplayColor");
40     String color = "red";
41     TrafficLight instance = new TrafficLight();
42     Color expectedResult = Color.RED;
43     Color result = instance.getDisplayColor(color);
44     assertEquals(expected: expectedResult, actual: result);
45 }
46
47 }
48
49
50
```

Test Case Output:

Test Results x

trafficlight.TrafficLightIT x

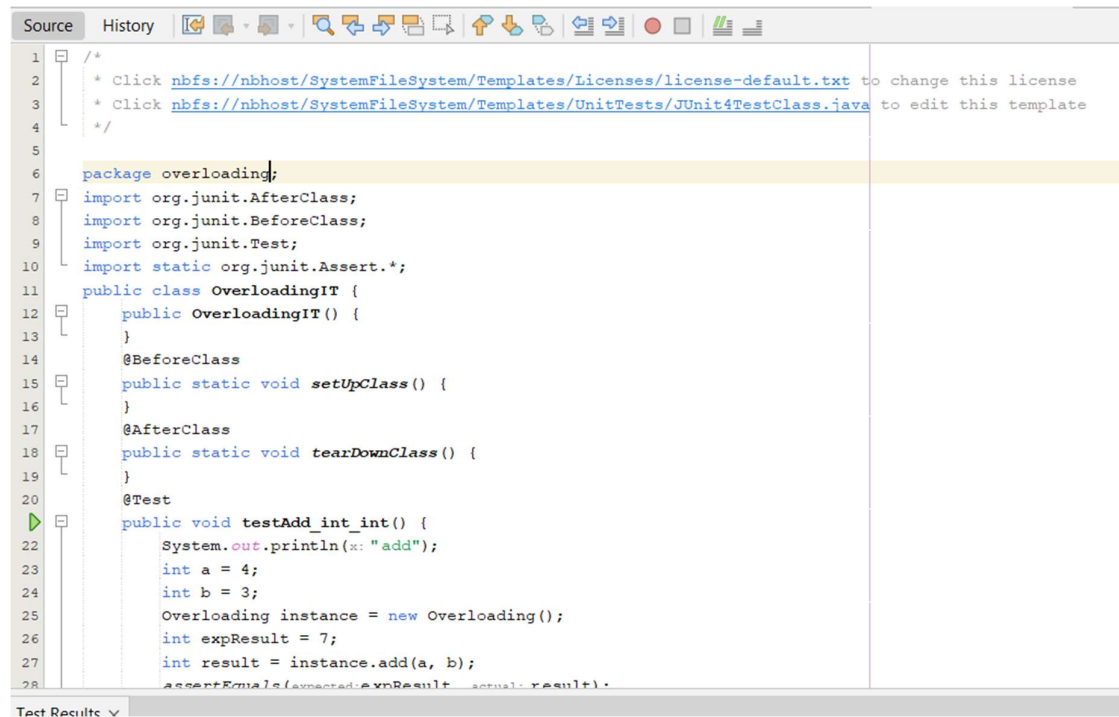
Tests passed: 100.00 %

Both tests passed. (0.17 s)

getDisplayColor
getMessage

Q2. Demonstrate test-driven development through a java file, Overloading.java, which properly tests the Overloading.java class to ensure that Method Overloading modules are working properly.

Answer:



```
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3   * Click nbfs://nbhost/SystemFileSystem/Templates/UnitTests/JUnit4TestClass.java to edit this template
4   */
5
6  package overloading;
7
8  import org.junit.AfterClass;
9  import org.junit.BeforeClass;
10 import org.junit.Test;
11 import static org.junit.Assert.*;
12
13 public class OverloadingIT {
14     public OverloadingIT() {
15     }
16
17     @BeforeClass
18     public static void setUpClass() {
19     }
20
21     @AfterClass
22     public static void tearDownClass() {
23     }
24
25     @Test
26     public void testAdd_int_int() {
27         System.out.println("add");
28         int a = 4;
29         int b = 3;
30         Overloading instance = new Overloading();
31         int expectedResult = 7;
32         int result = instance.add(a, b);
33         assertEquals("expected: " + expectedResult + " actual: " + result);
34     }
35 }
```

```
Source History
18 public static void testAddClass() {
19 }
20 @Test
21 public void testAdd_int_int() {
22     System.out.println("add");
23     int a = 4;
24     int b = 3;
25     Overloading instance = new Overloading();
26     int expectedResult = 7;
27     int result = instance.add(a, b);
28     assertEquals(expected: expectedResult, actual: result);
29 }
30 @Test
31 public void testAdd_double_double() {
32     System.out.println("add");
33     double a = 4.2;
34     double b = 2.0;
35     Overloading instance = new Overloading();
36     double expectedResult = 6.2;
37     double result = instance.add(a, b);
38     assertEquals(expected: expectedResult, actual: result, delta: 0);
39 }
40 }
41
42
```

Test Case Output:

Test Results ×

trafficlight.TrafficLightIT × overloading.OverloadingIT ×

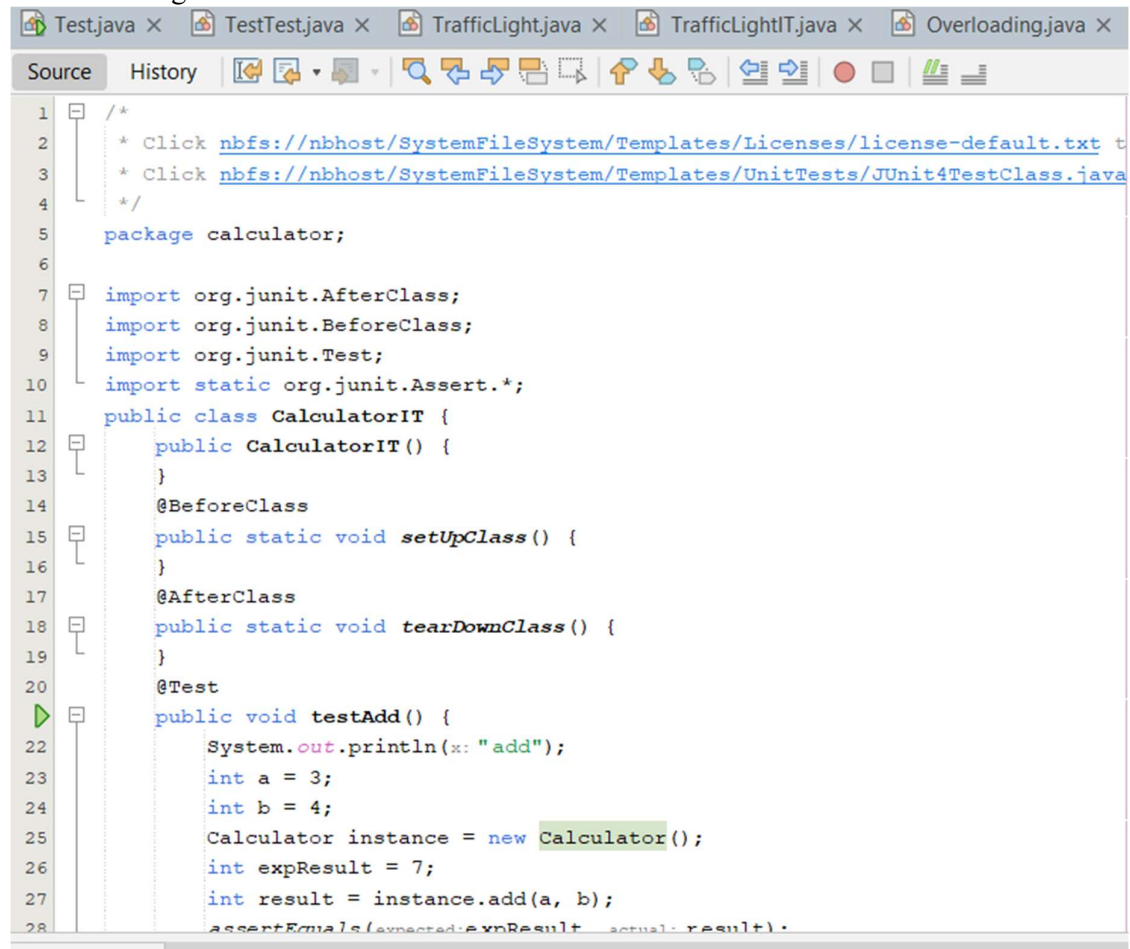
Tests passed: 100.00 %

Both tests passed. (0.061 s)

add

add

Q3. To study and implement unit testing for a Java program that performs addition of two numbers using JUnit framework.



The screenshot shows an IDE with several tabs open: Test.java, TestTest.java, TrafficLight.java, TrafficLightIT.java, and Overloading.java. The 'Source' tab is active, displaying the code for TrafficLightIT.java. The code is a JUnit test for a Calculator class. It includes package declarations, imports for JUnit classes, and a public class CalculatorIT. Inside CalculatorIT, there is a constructor, a @BeforeClass annotated static method setUpClass(), a @AfterClass annotated static method tearDownClass(), and a @Test annotated method testAdd(). The testAdd() method prints 'add', initializes variables a=3 and b=4, creates a Calculator instance, calculates the sum (7), and asserts the result.

```
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
3   * Click nbfs://nbhost/SystemFileSystem/Templates/UnitTests/JUnit4TestClass.java
4   */
5  package calculator;
6
7  import org.junit.AfterClass;
8  import org.junit.BeforeClass;
9  import org.junit.Test;
10 import static org.junit.Assert.*;
11 public class CalculatorIT {
12     public CalculatorIT() {
13     }
14     @BeforeClass
15     public static void setUpClass() {
16     }
17     @AfterClass
18     public static void tearDownClass() {
19     }
20     @Test
21     public void testAdd() {
22         System.out.println(x: "add");
23         int a = 3;
24         int b = 4;
25         Calculator instance = new Calculator();
26         int expResult = 7;
27         int result = instance.add(a, b);
28         assertEquals("expected: expResult = 7, actual: result = ", expResult, result);
29     }
30 }
```

```
Source History
19 }
20 @Test
21 public void testAdd() {
22     System.out.println("add");
23     int a = 3;
24     int b = 4;
25     Calculator instance = new Calculator();
26     int expectedResult = 7;
27     int result = instance.add(a, b);
28     assertEquals(expected: expectedResult, actual: result);
29 }
30 @Test
31 public void testMul() {
32     System.out.println("mul");
33     int a = 5;
34     int b = 2;
35     Calculator instance = new Calculator();
36     int expectedResult = 10;
37     int result = instance.mul(a, b);
38     assertEquals(expected: expectedResult, actual: result);
39 }
40 }
41
42
```

Test case Output:

Test Results x

trafficlight.TrafficLightIT x overloading.OverloadingIT x calculator.CalculatorIT x

▶

Tests passed: 100.00 %

⌵

Both tests passed. (0.06 s)

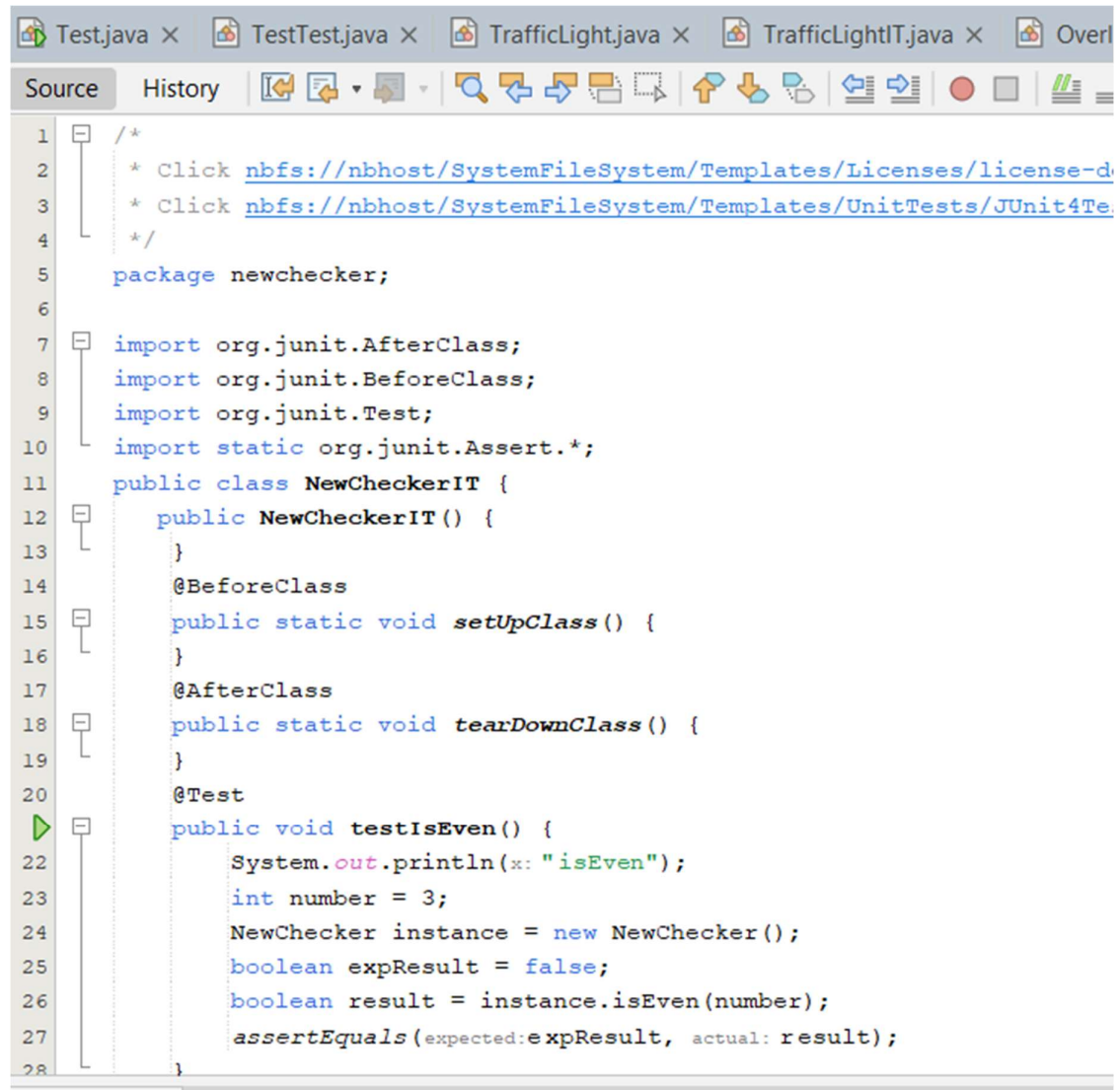
✓

>>

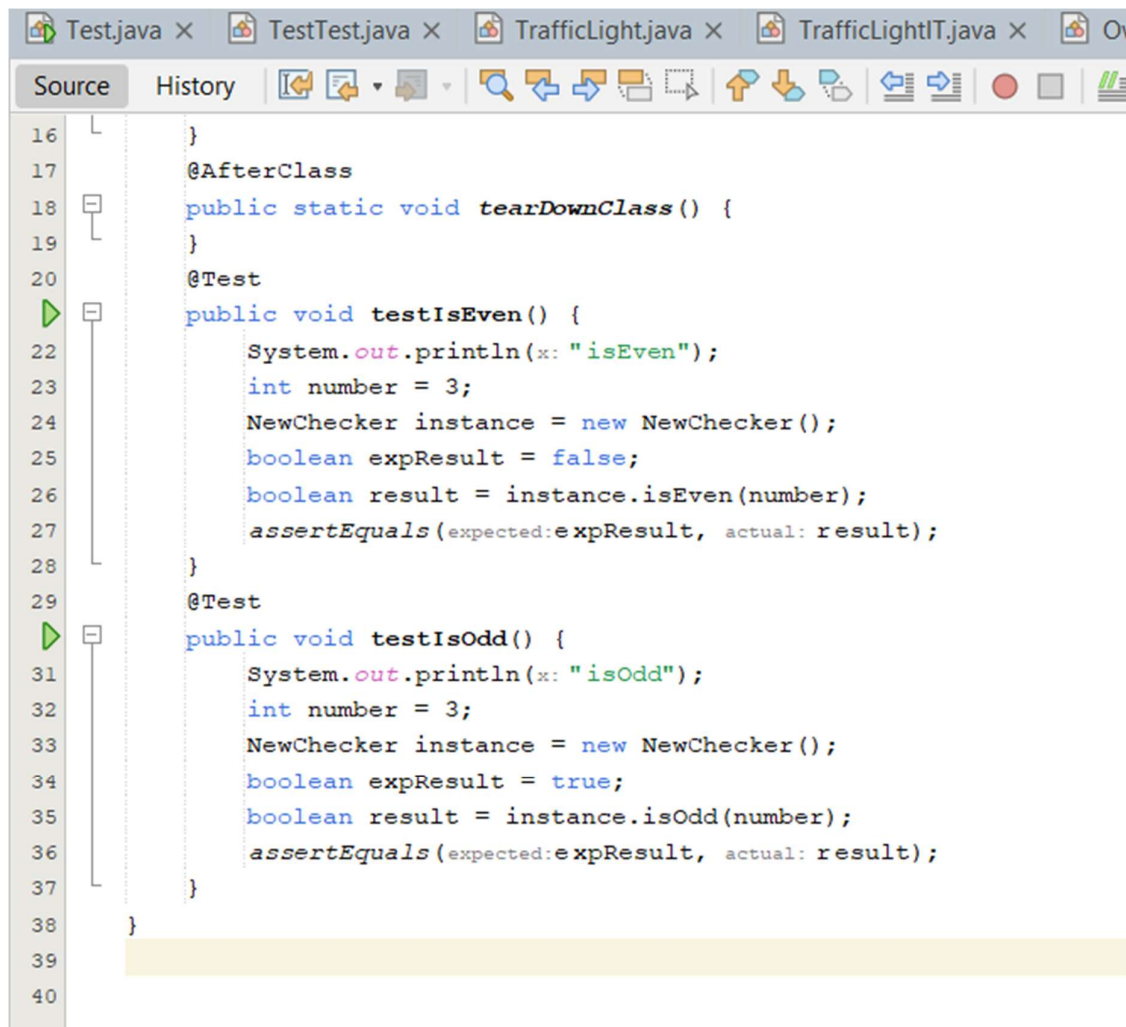
add
mul

Output

Q4. To design and test a Java program that checks whether a given number is even or odd using JUnit assertion methods.

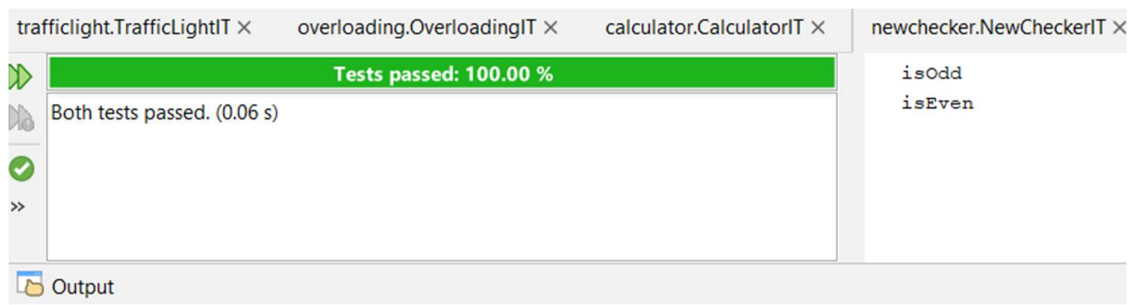


```
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-d
3   * Click nbfs://nbhost/SystemFileSystem/Templates/UnitTests/JUnit4Te
4   */
5   package newchecker;
6
7   import org.junit.AfterClass;
8   import org.junit.BeforeClass;
9   import org.junit.Test;
10  import static org.junit.Assert.*;
11
12  public class NewCheckerIT {
13      public NewCheckerIT() {
14      }
15      @BeforeClass
16      public static void setUpClass() {
17      }
18      @AfterClass
19      public static void tearDownClass() {
20      }
21      @Test
22      public void testIsEven() {
23          System.out.println(x: "isEven");
24          int number = 3;
25          NewChecker instance = new NewChecker();
26          boolean expResult = false;
27          boolean result = instance.isEven(number);
28          assertEquals(expected:expResult, actual: result);
29      }
30  }
```



```
16 }
17 @AfterClass
18 public static void tearDownClass() {
19 }
20 @Test
21 public void testIsEven() {
22     System.out.println("isEven");
23     int number = 3;
24     NewChecker instance = new NewChecker();
25     boolean expResult = false;
26     boolean result = instance.isEven(number);
27     assertEquals("expected:expResult, actual: result");
28 }
29 @Test
30 public void testIsOdd() {
31     System.out.println("isOdd");
32     int number = 3;
33     NewChecker instance = new NewChecker();
34     boolean expResult = true;
35     boolean result = instance.isOdd(number);
36     assertEquals("expected:expResult, actual: result");
37 }
38 }
39
40
```

Test Case Output:



trafficlight.TrafficLightIT ×	overloading.OverloadingIT ×	calculator.CalculatorIT ×	newchecker.NewCheckerIT ×
Tests passed: 100.00 %			isOdd
Both tests passed. (0.06 s)			isEven

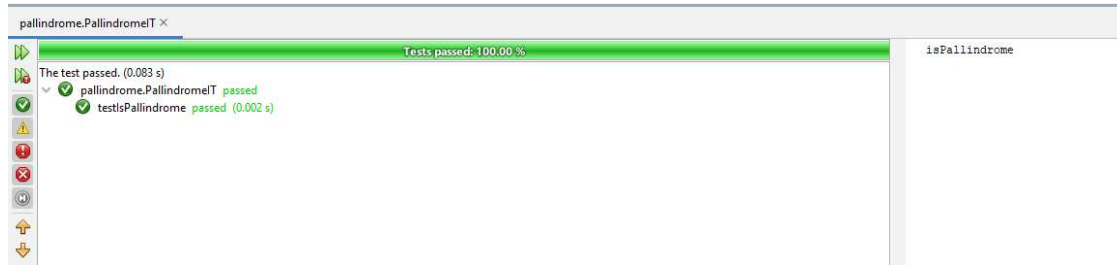
Output

Q5. Implement and test a Java program that verifies whether a given string is a **palindrome** using **JUnit** test cases and assertions.

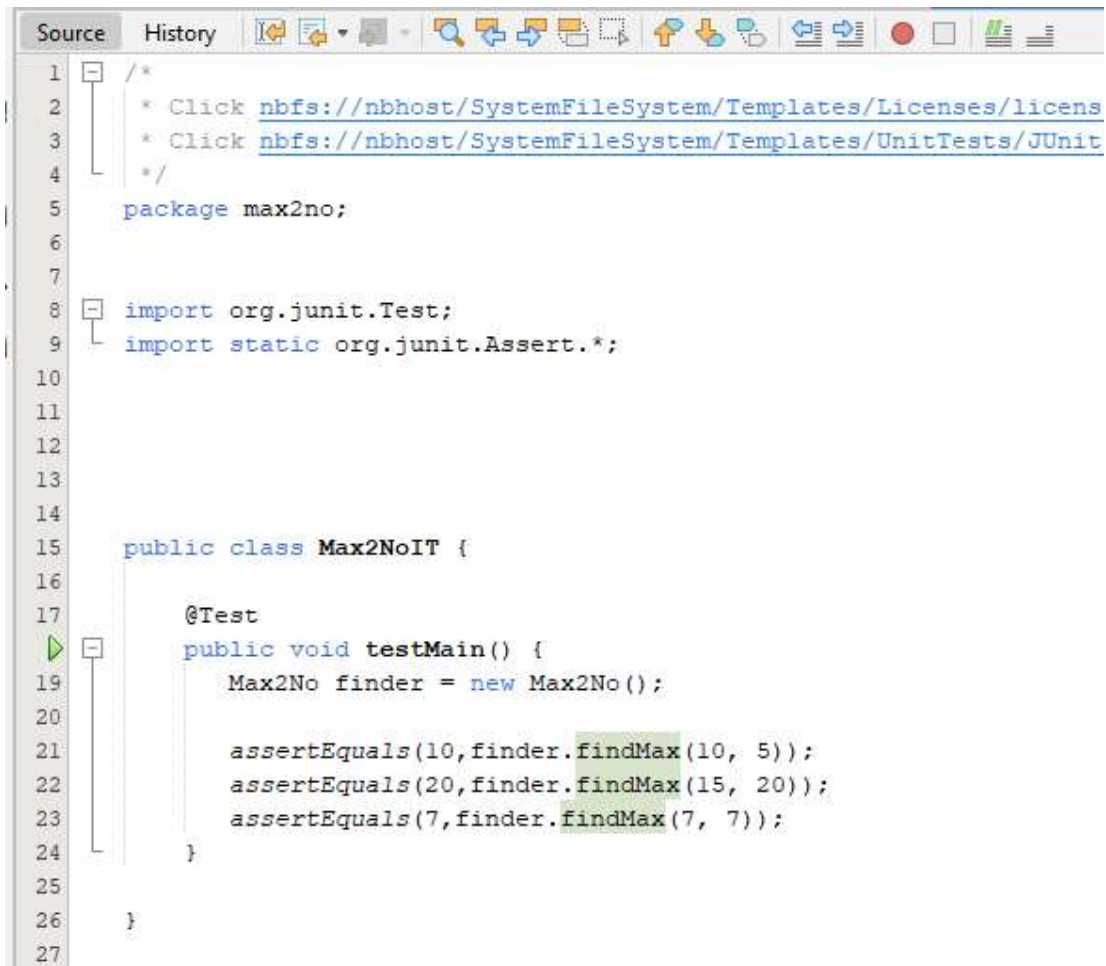
A screenshot of an IDE window showing a Java source file. The window has a title bar with 'Source' and 'History' tabs. The code is for a package named 'pallindrome' and a class named 'PallindromeIT'. It includes imports for JUnit's AfterClass, Test, and Assert classes. The class has a constructor, an @AfterClass method 'tearDownClass', and a @Test method 'testIsPallindrome'. The test method prints 'isPallindrome', sets a string 'madam' to a variable 'str', sets a boolean 'expResult' to true, calls 'Pallindrome.isPallindrome(str)', and asserts that 'expResult' equals the return value. The code is numbered from 1 to 42 on the left margin.

```
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-defau
3   * Click nbfs://nbhost/SystemFileSystem/Templates/UnitTests/JUnit4TestCl
4   */
5  package pallindrome;
6
7  import org.junit.AfterClass;
8  import org.junit.Test;
9  import static org.junit.Assert.*;
10
11
12
13
14
15  public class PallindromeIT {
16
17      public PallindromeIT() {
18      }
19
20      @AfterClass
21      public static void tearDownClass() {
22      }
23
24      /**
25       * Test of main method, of class Palindrome.
26       */
27
28
29      /**
30       * Test of isPallindrome method, of class Pallindrome.
31       */
32      @Test
33      public void testIsPallindrome() {
34          System.out.println("isPallindrome");
35          String str = "madam";
36          boolean expResult =true;
37          boolean result = Pallindrome.isPallindrome(str);
38          assertEquals(expResult, result);
39      }
40
41  }
42
```

Test Case Output:

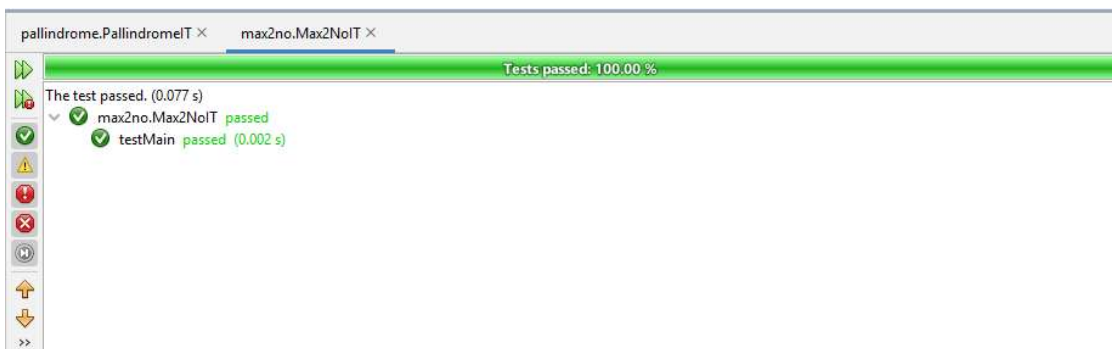


Q6. Demonstrate test-driven development through a Java program to find the maximum of two numbers and test the logic using JUnit framework.



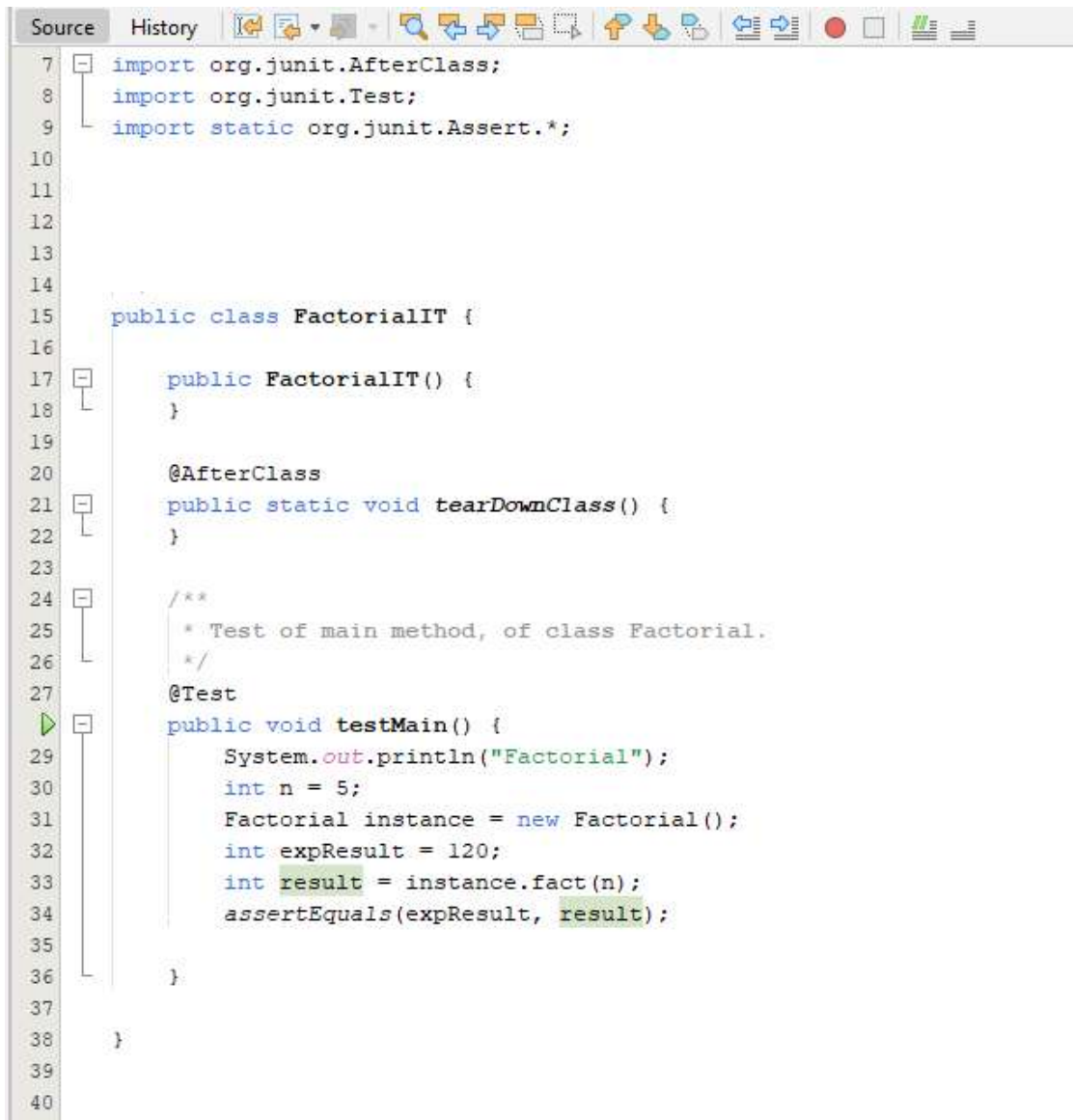
```
1  /*
2   * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/licens
3   * Click nbfs://nbhost/SystemFileSystem/Templates/UnitTests/JUnit
4   */
5   package max2no;
6
7
8   import org.junit.Test;
9   import static org.junit.Assert.*;
10
11
12
13
14
15   public class Max2NoIT {
16
17       @Test
18       public void testMain() {
19           Max2No finder = new Max2No();
20
21           assertEquals(10,finder.findMax(10, 5));
22           assertEquals(20,finder.findMax(15, 20));
23           assertEquals(7,finder.findMax(7, 7));
24       }
25
26   }
27
```

Test Case Output:



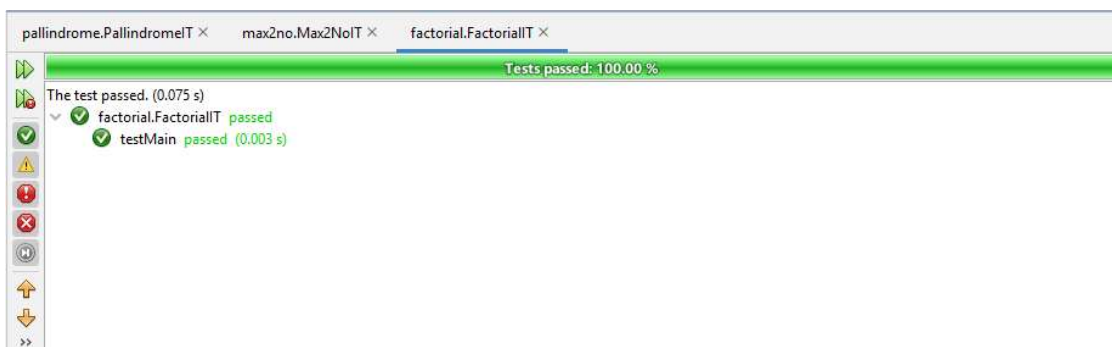
```
pallindrome.PallindromeIT x max2no.Max2NoIT x
Tests passed: 100.00 %
The test passed. (0.077 s)
  max2no.Max2NoIT passed
    testMain passed (0.002 s)
```

Q7. To create, demonstrate, and implement a Java program to calculate the factorial of a number and verify correctness using JUnit testing.



```
Source History
7 import org.junit.AfterClass;
8 import org.junit.Test;
9 import static org.junit.Assert.*;
10
11
12
13
14
15 public class FactorialIT {
16
17     public FactorialIT() {
18     }
19
20     @AfterClass
21     public static void tearDownClass() {
22     }
23
24     /**
25      * Test of main method, of class Factorial.
26      */
27     @Test
28     public void testMain() {
29         System.out.println("Factorial");
30         int n = 5;
31         Factorial instance = new Factorial();
32         int expResult = 120;
33         int result = instance.fact(n);
34         assertEquals(expResult, result);
35     }
36 }
37
38
39
40
```

Test Case Output:



```
pallindrome.PallindromeIT x max2no.Max2NoIT x factorial.FactorialIT x
Tests passed: 100.00 %
The test passed. (0.075 s)
  ✓ factorial.FactorialIT passed
    ✓ testMain passed (0.003 s)
```

Piyush Goel (09311604425)

Q8 Demonstrate test-driven development through a simple project called reverse in Net Beans. Reverse a string and test the result using JUnit unit testing methods.

```
Source History
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5  package avneetst;
6
7
8
9
10
11  public class Reverse {
12
13      public String reverseString(String input) {
14          if (input == null) {
15              return null;
16          }
17
18          String reversed = "";
19          for (int i = input.length() - 1; i >= 0; i--) {
20              reversed = reversed + input.charAt(index: i);
21          }
22
23          return reversed;
24      }
25
26      public static void main(String[] args) {
27
28      }
29  }
```

//ReverseIT.java

```
~/
public class ReverseIT {

    Reverse rev = new Reverse();

    @Test
    public void testReverseNormalString() {
        assertEquals(expected: "olleh", actual: rev.reverseString(input: "hello"));
    }

    @Test
    public void testReverseSingleCharacter() {
        assertEquals(expected: "a", actual: rev.reverseString(input: "a"));
    }

    @Test
    public void testReverseEmptyString() {
        assertEquals(expected: "", actual: rev.reverseString(input: ""));
    }

    @Test
    public void testReverseWithSpaces() {
        assertEquals(expected: "dlrow olleh", actual: rev.reverseString(input: "hello world"));
    }

    @Test
    public void testNullString() {
        assertEquals(expected: null, actual: rev.reverseString(input: null));
    }
}
```

Test Case Output:

Output - AvneetST (test)

Test Results ×

avneetst.ReverseIT ×

▶▶

▶▶

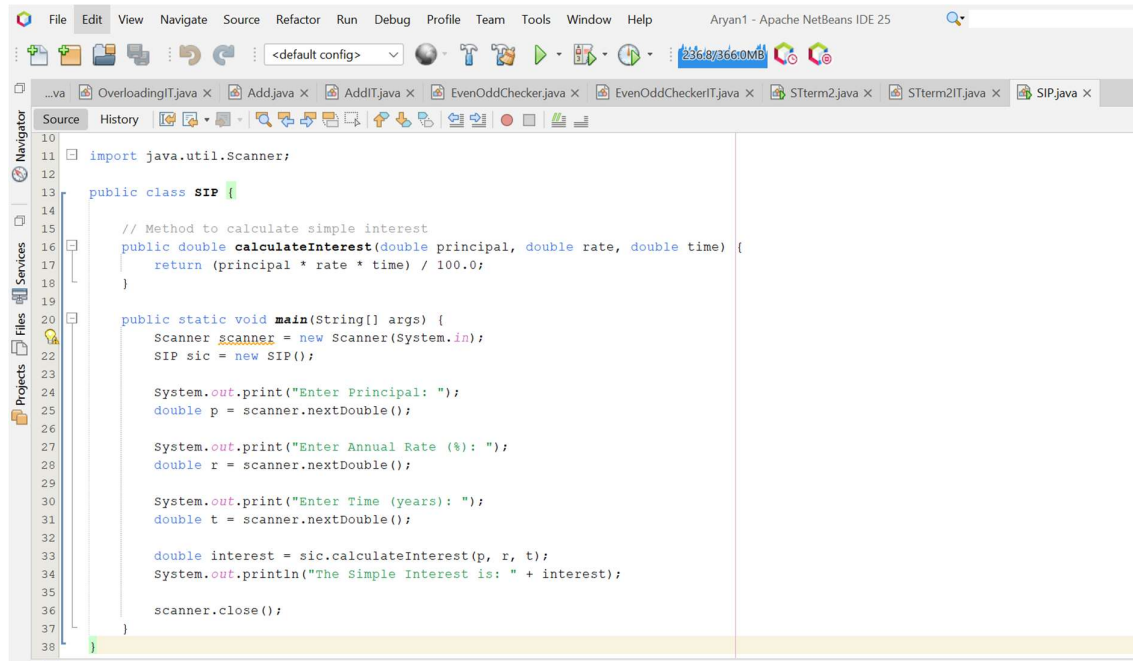
✓

Tests passed: 100.00 %

All 5 tests passed. (0.077 s)

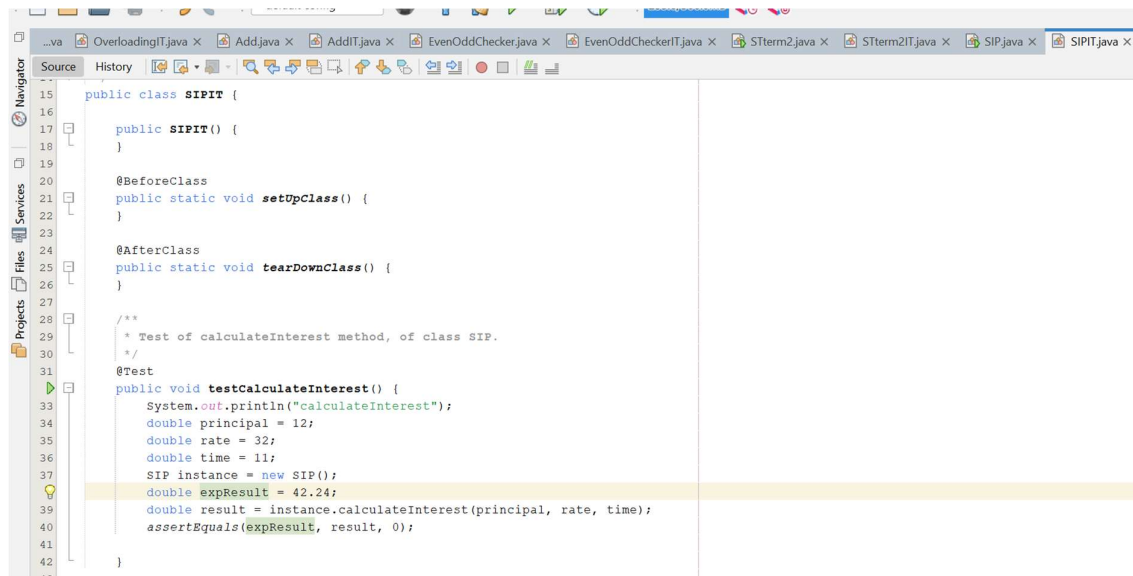
> ✓ avneetst.ReverseIT passed

Q9. Create and implement a Java program to calculate simple interest and verify the computation using JUnit test cases.



```
10 import java.util.Scanner;
11
12
13 public class SIP {
14
15     // Method to calculate simple interest
16     public double calculateInterest(double principal, double rate, double time) {
17         return (principal * rate * time) / 100.0;
18     }
19
20     public static void main(String[] args) {
21         Scanner scanner = new Scanner(System.in);
22         SIP sic = new SIP();
23
24         System.out.print("Enter Principal: ");
25         double p = scanner.nextDouble();
26
27         System.out.print("Enter Annual Rate (%): ");
28         double r = scanner.nextDouble();
29
30         System.out.print("Enter Time (years): ");
31         double t = scanner.nextDouble();
32
33         double interest = sic.calculateInterest(p, r, t);
34         System.out.println("The Simple Interest is: " + interest);
35
36         scanner.close();
37     }
38 }
```

//SIPIT.java



```
15 public class SIPIT {
16
17     public SIPIT() {
18     }
19
20     @BeforeClass
21     public static void setUpClass() {
22     }
23
24     @AfterClass
25     public static void tearDownClass() {
26     }
27
28     /**
29      * Test of calculateInterest method, of class SIP.
30      */
31     @Test
32     public void testCalculateInterest() {
33         System.out.println("calculateInterest");
34         double principal = 12;
35         double rate = 32;
36         double time = 11;
37         SIP instance = new SIP();
38         double expectedResult = 42.24;
39         double result = instance.calculateInterest(principal, rate, time);
40         assertEquals(expectedResult, result, 0);
41     }
42 }
```

Test Case Output:

Services

Files

Projects

```
33 public void testCalculateInterest() {
34     System.out.println("calculateInterest");
35     double principal = 12;
36     double rate = 32;
37     double time = 11;
38     SIP instance = new SIP();
39     double expectedResult = 42.24;
40     double result = instance.calculateInterest(principal, rate, time);
41     assertEquals(expectedResult, result, 0);
42 }
43
44
45
```

Test Results

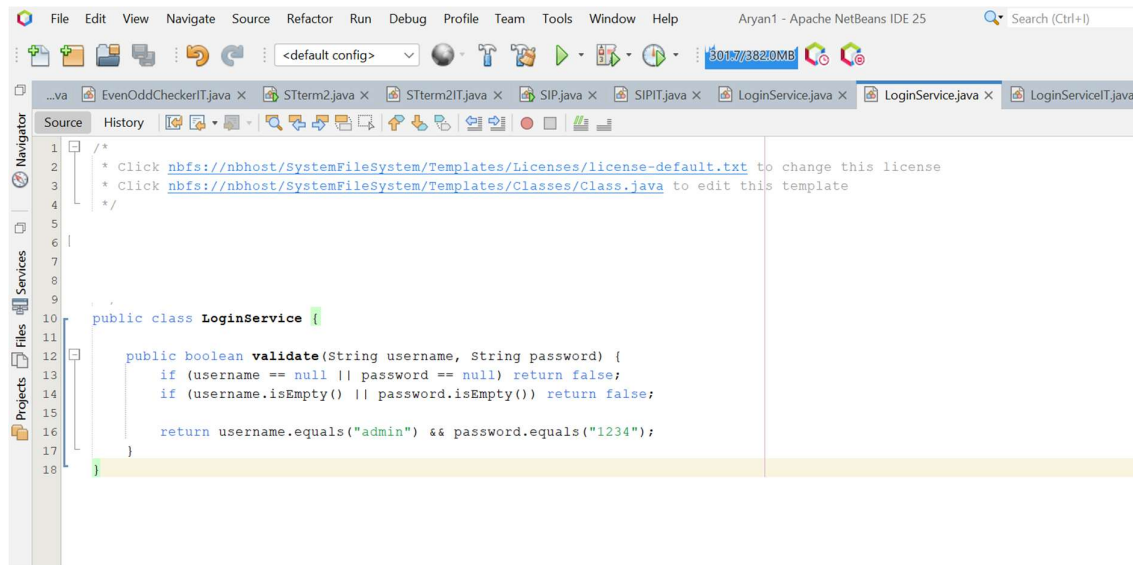
SIPIT X

Tests passed: 100.00 %

calculateInterest

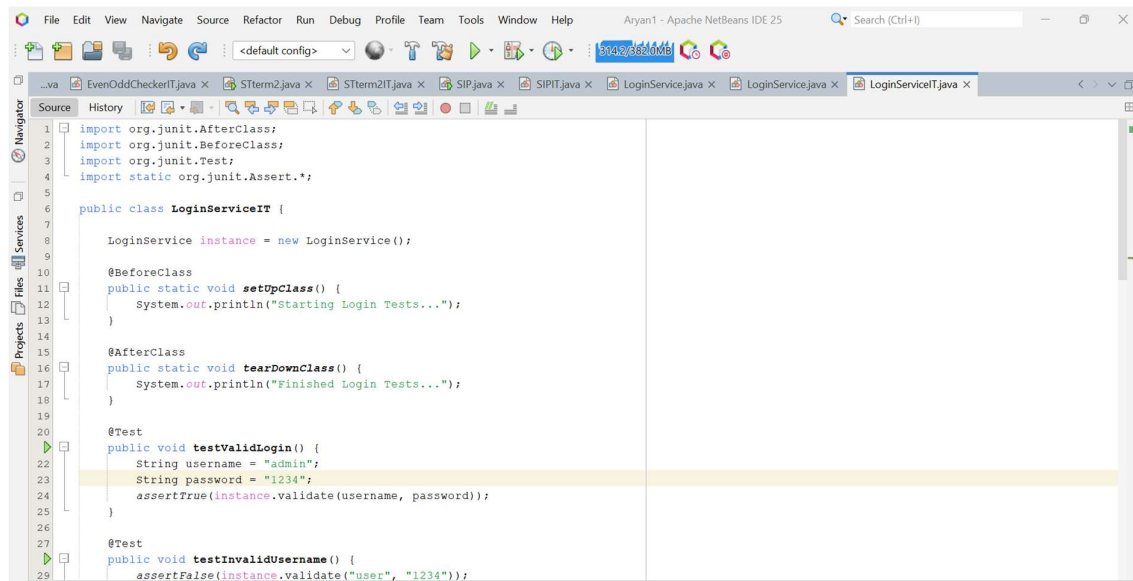
The test passed. (0.12 s)

Q10. Construct proper test cases for a Program to validate user login credentials and test different input scenarios using JUnit testing



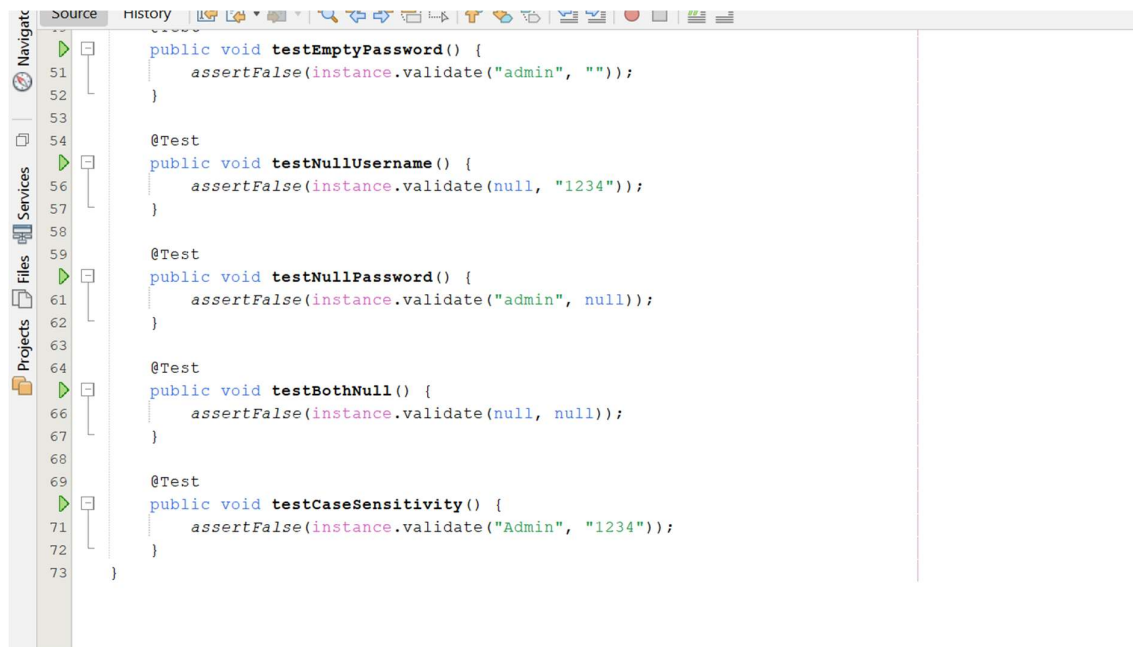
```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5
6
7
8
9
10 public class LoginService {
11
12     public boolean validate(String username, String password) {
13         if (username == null || password == null) return false;
14         if (username.isEmpty() || password.isEmpty()) return false;
15
16         return username.equals("admin") && password.equals("1234");
17     }
18 }
```

//LoginServiceIT.java



```
1  import org.junit.AfterClass;
2  import org.junit.BeforeClass;
3  import org.junit.Test;
4  import static org.junit.Assert.*;
5
6  public class LoginServiceIT {
7
8      LoginService instance = new LoginService();
9
10     @BeforeClass
11     public static void setUpClass() {
12         System.out.println("Starting Login Tests...");
13     }
14
15     @AfterClass
16     public static void tearDownClass() {
17         System.out.println("Finished Login Tests...");
18     }
19
20     @Test
21     public void testValidLogin() {
22         String username = "admin";
23         String password = "1234";
24         assertTrue(instance.validate(username, password));
25     }
26
27     @Test
28     public void testInvalidUsername() {
29         assertFalse(instance.validate("user", "1234"));
30     }
31
32     @Test
33     public void testInvalidPassword() {
34         assertFalse(instance.validate("admin", "wrong"));
35     }
36
37     @Test
38     public void testInvalidBoth() {
39         assertFalse(instance.validate("user", "wrong"));
40     }
41
42     @Test
43     public void testEmptyUsername() {
44         assertFalse(instance.validate("", "1234"));
45     }
46
47     @Test
48     public void testEmptyPassword() {
49         assertFalse(instance.validate("admin", ""));
50     }
51
52 }
```

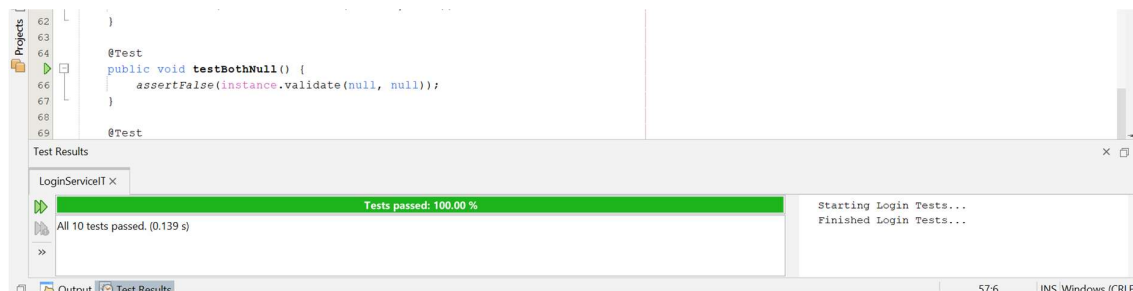
Piyush Goel (09311604425)



The screenshot shows an IDE window with a source code editor. On the left, there is a 'Projects' sidebar with icons for 'Navigate', 'Services', 'Files', and 'Projects'. The main editor displays the following Java code:

```
51 public void testEmptyPassword() {  
52     assertFalse(instance.validate("admin", ""));  
53 }  
54  
55 @Test  
56 public void testNullUsername() {  
57     assertFalse(instance.validate(null, "1234"));  
58 }  
59  
60 @Test  
61 public void testNullPassword() {  
62     assertFalse(instance.validate("admin", null));  
63 }  
64  
65 @Test  
66 public void testBothNull() {  
67     assertFalse(instance.validate(null, null));  
68 }  
69  
70 @Test  
71 public void testCaseSensitivity() {  
72     assertFalse(instance.validate("Admin", "1234"));  
73 }
```

Test Case Output:

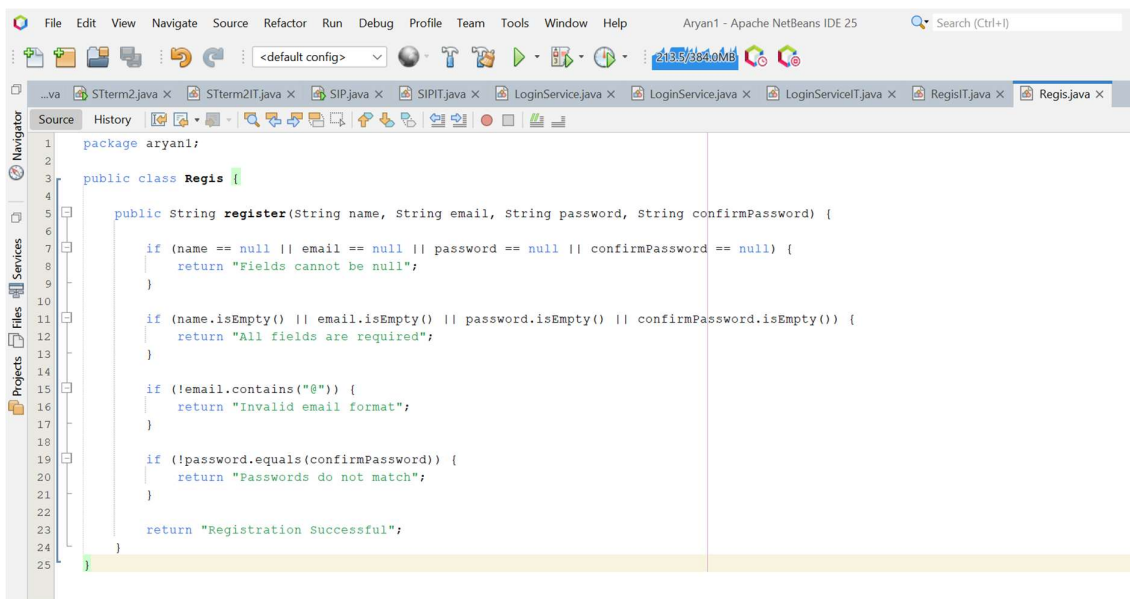


The screenshot shows the 'Test Results' window in an IDE. The window has a tab titled 'LoginServiceIT x'. The main area displays a green progress bar with the text 'Tests passed: 100.00 %'. Below the progress bar, it says 'All 10 tests passed. (0.139 s)'. On the right side of the window, there is a text area with the following content:

```
Starting Login Tests...  
Finished Login Tests...
```

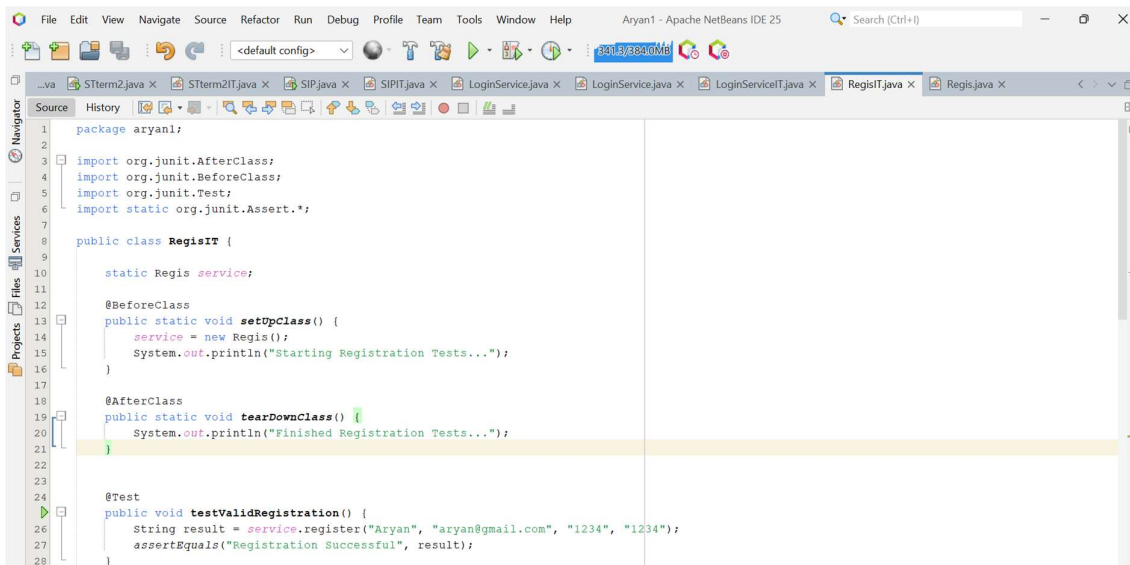
Q11. Construct proper test cases for a general registration form. The test scenarios can be as follows:

- i. Verify that the registration page loads successfully (all specified fields appear).
- ii. Verify that the required/mandatory fields are marked with * against the field.
- iii. Verify the page has both submit and cancel/reset buttons at the end.
- iv. Verify that clicking submit button after entering all the required fields, submits the data to the server.
- v. Verify that clicking cancels/reset button after entering all the required fields, cancels the submit request, and reset all the fields.
- vi. Verify that not filling the man



```
1 package aryan1;
2
3 public class Regis {
4
5     public String register(String name, String email, String password, String confirmPassword) {
6
7         if (name == null || email == null || password == null || confirmPassword == null) {
8             return "Fields cannot be null";
9         }
10
11         if (name.isEmpty() || email.isEmpty() || password.isEmpty() || confirmPassword.isEmpty()) {
12             return "All fields are required";
13         }
14
15         if (!email.contains("@")) {
16             return "Invalid email format";
17         }
18
19         if (!password.equals(confirmPassword)) {
20             return "Passwords do not match";
21         }
22
23         return "Registration Successful";
24     }
25 }
```

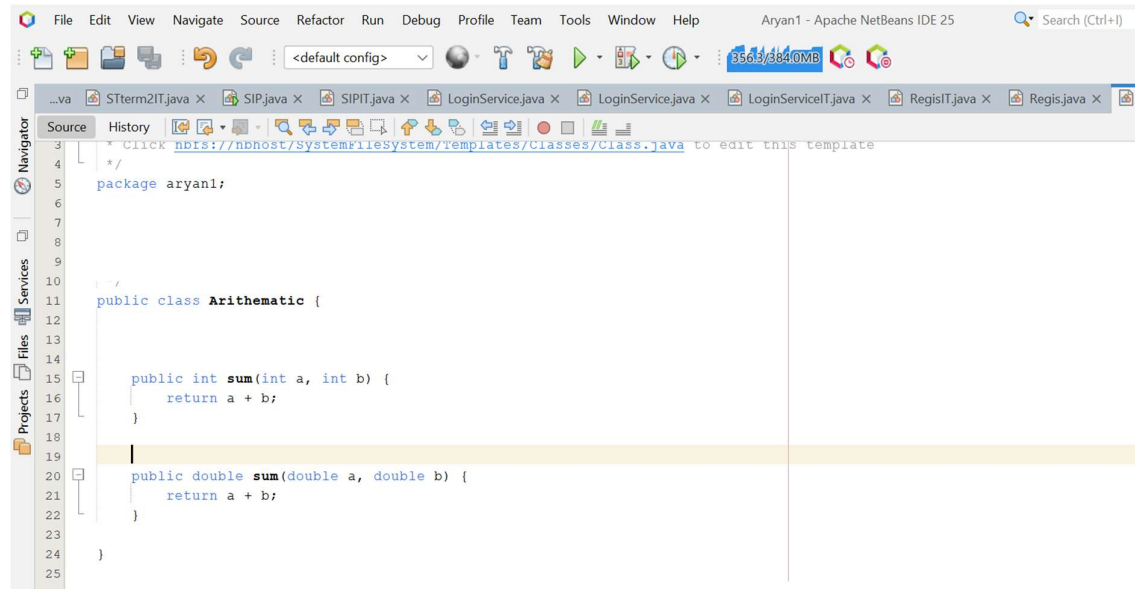
//RegisIT.java



```
1 package aryan1;
2
3 import org.junit.AfterClass;
4 import org.junit.BeforeClass;
5 import org.junit.Test;
6 import static org.junit.Assert.*;
7
8 public class RegisIT {
9
10     static Regis service;
11
12     @BeforeClass
13     public static void setUpClass() {
14         service = new Regis();
15         System.out.println("Starting Registration Tests...");
16     }
17
18     @AfterClass
19     public static void tearDownClass() {
20         System.out.println("Finished Registration Tests...");
21     }
22
23     @Test
24     public void testValidRegistration() {
25         String result = service.register("Aryan", "aryan@gmail.com", "1234", "1234");
26         assertEquals("Registration Successful", result);
27     }
28 }
```

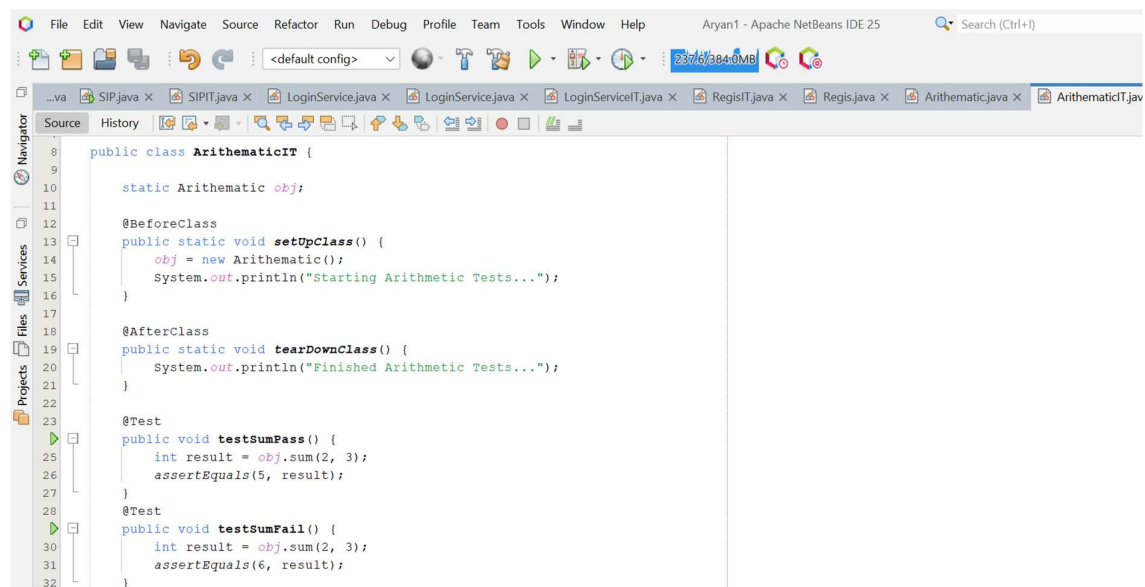
```
Source History
14 service = new Regis();
15 System.out.println("Starting Registration Tests...");
16 }
17
18 @AfterClass
19 public static void tearDownClass() {
20     System.out.println("Finished Registration Tests...");
21 }
22
23
24 @Test
25 public void testValidRegistration() {
26     String result = service.register("Aryan", "aryan@gmail.com", "1234", "1234");
27     assertEquals("Registration Successful", result);
28 }
29
30
31 @Test
32 public void testEmptyFields() {
33     String result = service.register("", "a@gmail.com", "1234", "1234");
34     assertEquals("All fields are required", result);
35 }
36
37
38 @Test
39 public void testInvalidEmail() {
40     String result = service.register("Aryan", "invalidEmail", "1234", "1234");
41     assertEquals("Invalid email format", result);
42 }
43
44
45
46
47
48
49
50
51
52 @Test
53 public void testNullValues() {
54     String result = service.register(null, null, null, null);
55     assertEquals("Fields cannot be null", result);
56 }
57
58 @Test
59 public void testMandatoryFieldsValidation() {
60     String result = service.register("", "", "", "");
61     assertEquals("All fields are required", result);
62 }
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2
```

Q12. In the class Arithmetic.java in Q3. Above add another sum() method. Also generate the corresponding sumTest() method through Junit. Execute the Arithmetic Test class for inputs in which test cases fails as well as inputs where test cases pass. Record both test cases in proper test case templates

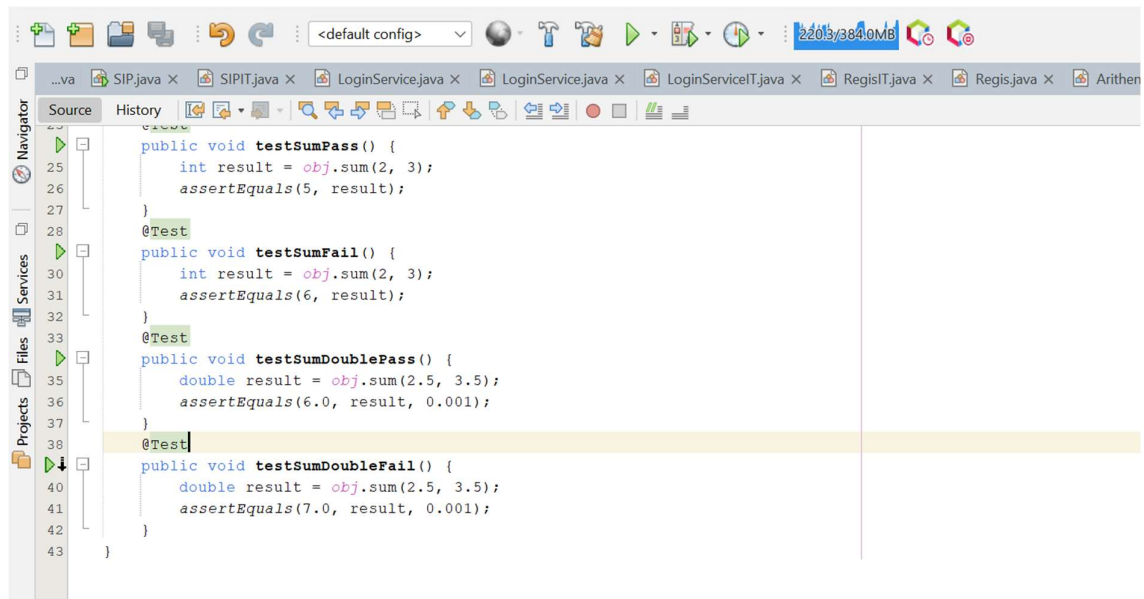


```
3  * Click https://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5  package aryan1;
6
7
8
9
10
11 public class Arithmetic {
12
13
14
15     public int sum(int a, int b) {
16         return a + b;
17     }
18
19
20     public double sum(double a, double b) {
21         return a + b;
22     }
23
24
25 }
```

//ArithmeticIT.java



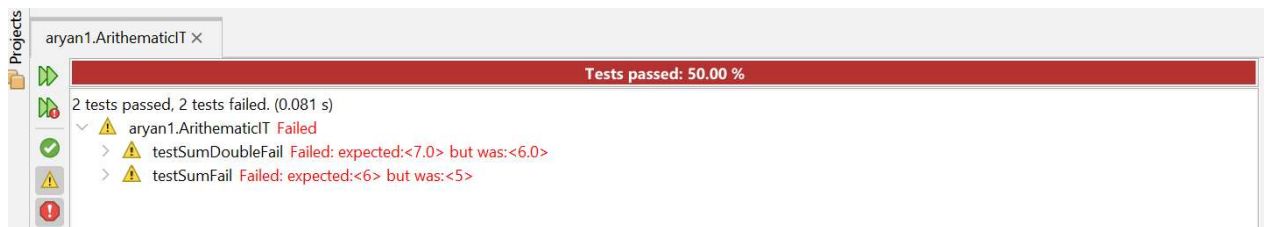
```
8 public class ArithmeticIT {
9
10     static Arithmetic obj;
11
12     @BeforeClass
13     public static void setUpClass() {
14         obj = new Arithmetic();
15         System.out.println("Starting Arithmetic Tests...");
16     }
17
18     @AfterClass
19     public static void tearDownClass() {
20         System.out.println("Finished Arithmetic Tests...");
21     }
22
23     @Test
24     public void testSumPass() {
25         int result = obj.sum(2, 3);
26         assertEquals(5, result);
27     }
28
29     @Test
30     public void testSumFail() {
31         int result = obj.sum(2, 3);
32         assertEquals(6, result);
33     }
34 }
```



The screenshot shows an IDE with several tabs open: SIP.java, SIPIT.java, LoginService.java, LoginServiceIT.java, RegisIT.java, and Arithen. The main editor displays the source code for a test class. The code includes four test methods: testSumPass, testSumFail, testSumDoublePass, and testSumDoubleFail. The testSumFail method is currently selected and highlighted in yellow. The code is as follows:

```
public void testSumPass() {  
    int result = obj.sum(2, 3);  
    assertEquals(5, result);  
}  
  
@Test  
public void testSumFail() {  
    int result = obj.sum(2, 3);  
    assertEquals(6, result);  
}  
  
@Test  
public void testSumDoublePass() {  
    double result = obj.sum(2.5, 3.5);  
    assertEquals(6.0, result, 0.001);  
}  
  
@Test  
public void testSumDoubleFail() {  
    double result = obj.sum(2.5, 3.5);  
    assertEquals(7.0, result, 0.001);  
}
```

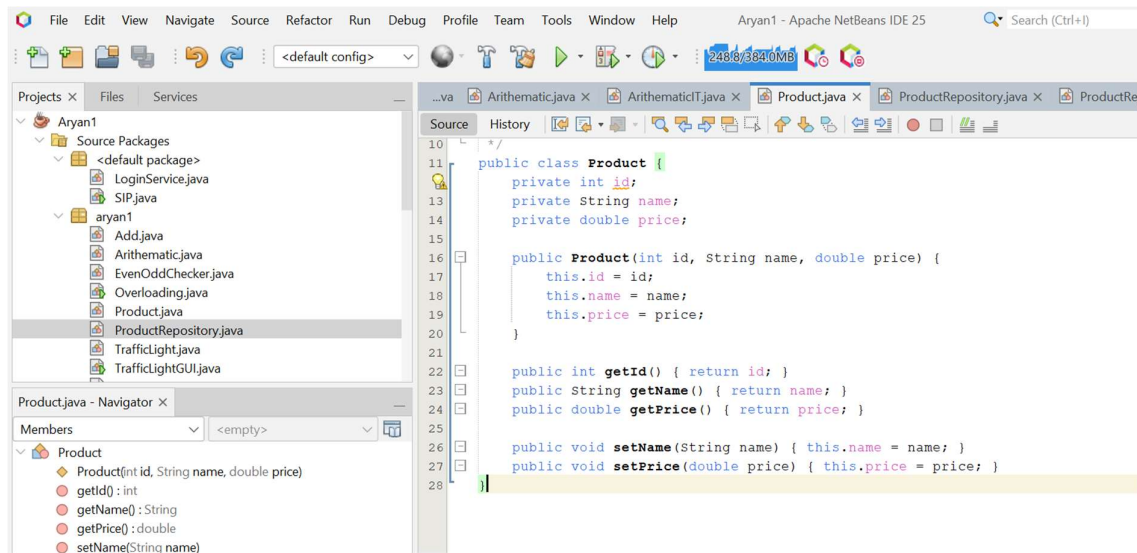
Test Case Output:



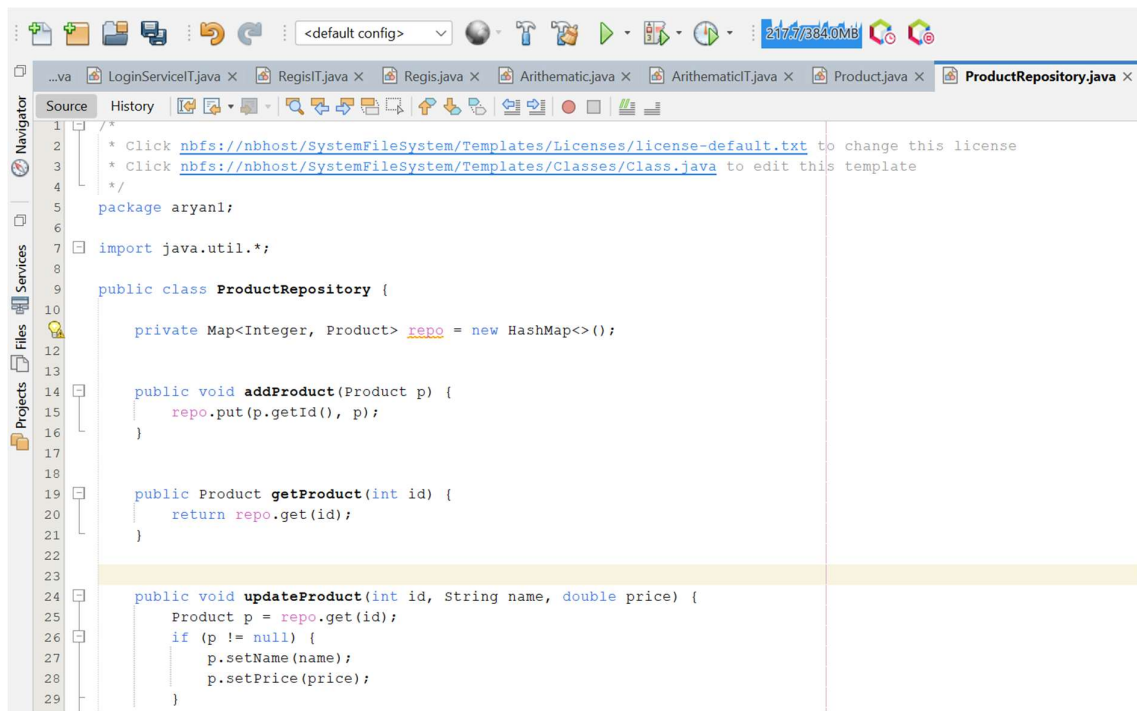
The screenshot shows the Test Results window for the test class aryan1.ArithmeticIT. The window displays the following information:

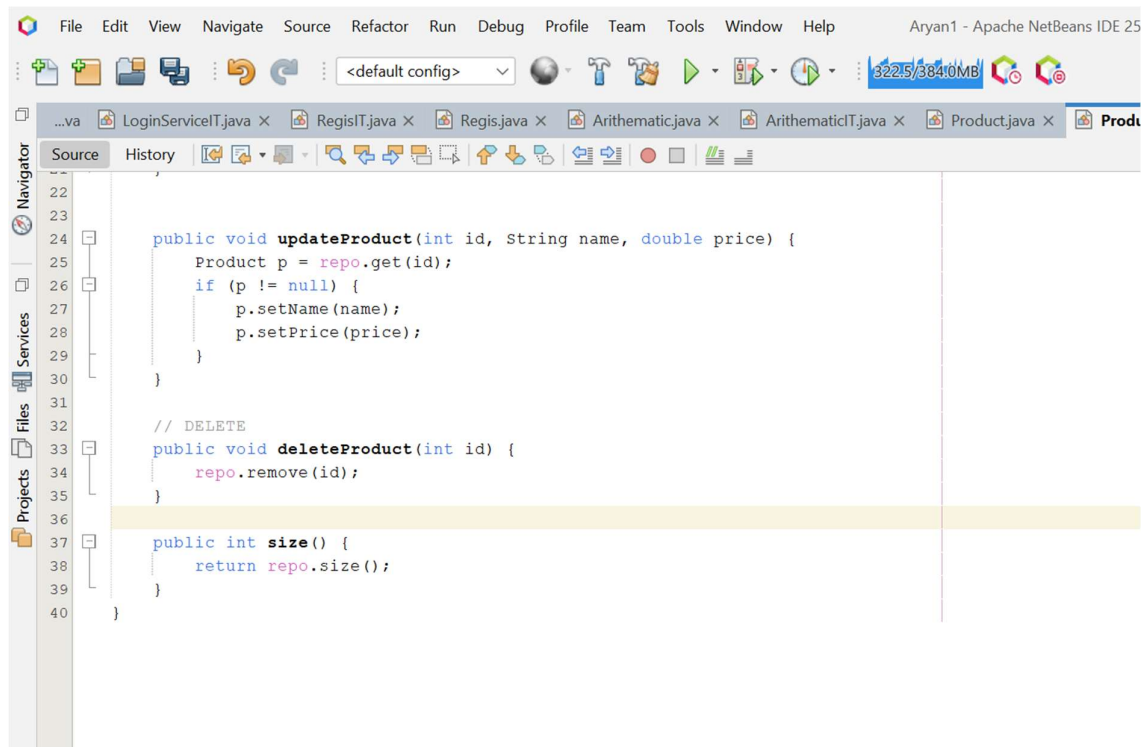
- Tests passed: 50.00 %
- 2 tests passed, 2 tests failed. (0.081 s)
- Test Results:
 - testSumDoubleFail Failed: expected:<7.0> but was:<6.0>
 - testSumFail Failed: expected:<6> but was:<5>

Q13. Demonstrate test driven development through a project Project Repository. The project should contain a class ProductRepository with appropriate methods for CRUD operations on a ProductRepository instance. Through JUnit create corresponding Test classes for each class. Aggregate these different tests in a test suite and execute the same.



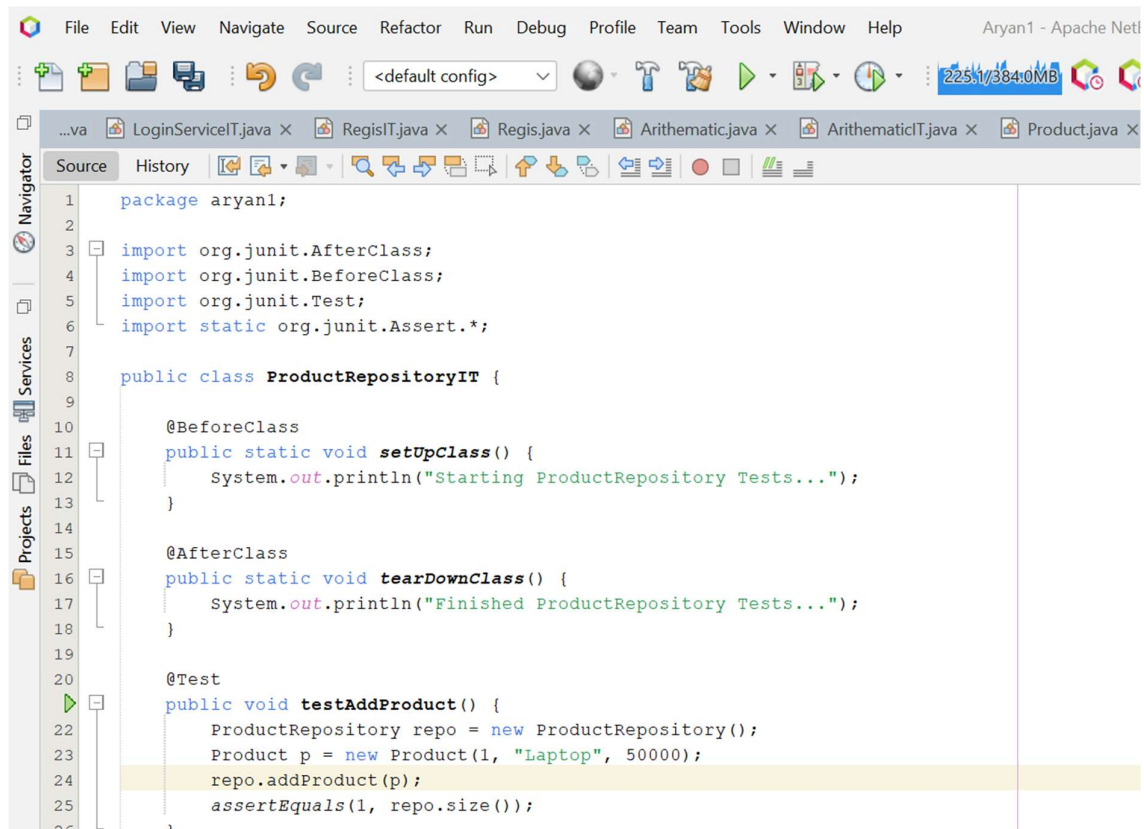
//ProductRepository.java





```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help Aryan1 - Apache NetBeans IDE 25
... LoginServiceIT.java x RegisIT.java x Regis.java x Arithmetic.java x ArithmeticIT.java x Product.java x
Source History
22
23
24 public void updateProduct(int id, String name, double price) {
25     Product p = repo.get(id);
26     if (p != null) {
27         p.setName(name);
28         p.setPrice(price);
29     }
30 }
31
32 // DELETE
33 public void deleteProduct(int id) {
34     repo.remove(id);
35 }
36
37 public int size() {
38     return repo.size();
39 }
40 }
```

//ProductRepositoryIT.java



```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help Aryan1 - Apache NetBeans IDE 25
... LoginServiceIT.java x RegisIT.java x Regis.java x Arithmetic.java x ArithmeticIT.java x Product.java x
Source History
1 package aryan1;
2
3 import org.junit.AfterClass;
4 import org.junit.BeforeClass;
5 import org.junit.Test;
6 import static org.junit.Assert.*;
7
8 public class ProductRepositoryIT {
9
10     @BeforeClass
11     public static void setUpClass() {
12         System.out.println("Starting ProductRepository Tests...");
13     }
14
15     @AfterClass
16     public static void tearDownClass() {
17         System.out.println("Finished ProductRepository Tests...");
18     }
19
20     @Test
21     public void testAddProduct() {
22         ProductRepository repo = new ProductRepository();
23         Product p = new Product(1, "Laptop", 50000);
24         repo.addProduct(p);
25         assertEquals(1, repo.size());
26     }
27 }
```

```

16 public static void tearDownClass() {
17     System.out.println("Finished ProductRepository Tests...");
18 }
19
20 @Test
21 public void testAddProduct() {
22     ProductRepository repo = new ProductRepository();
23     Product p = new Product(1, "Laptop", 50000);
24     repo.addProduct(p);
25     assertEquals(1, repo.size());
26 }
27
28 @Test
29 public void testGetProduct() {
30     ProductRepository repo = new ProductRepository();
31     Product p = new Product(2, "Phone", 20000);
32     repo.addProduct(p);
33     assertEquals("Phone", repo.getProduct(2).getName());
34 }
35
36 @Test
37 public void testUpdateProduct() {
38     ProductRepository repo = new ProductRepository();
39     Product p = new Product(3, "Tablet", 15000);
40     repo.addProduct(p);
41     repo.updateProduct(3, "iPad", 30000);
42     assertEquals("iPad", repo.getProduct(3).getName());
43 }
44
45 @Test
46 public void testDeleteProduct() {
47     ProductRepository repo = new ProductRepository();
48     Product p = new Product(4, "Watch", 5000);
49     repo.addProduct(p);
50     repo.deleteProduct(4);
51     assertNull(repo.getProduct(4));
52 }
53 }

```

Test Case Output:

```

20 @Test
21 public void testAddProduct() {
22     ProductRepository repo = new ProductRepository();
23     Product p = new Product(1, "Laptop", 50000);
24     repo.addProduct(p);
25     assertEquals(1, repo.size());
26 }
27

```

Test Results

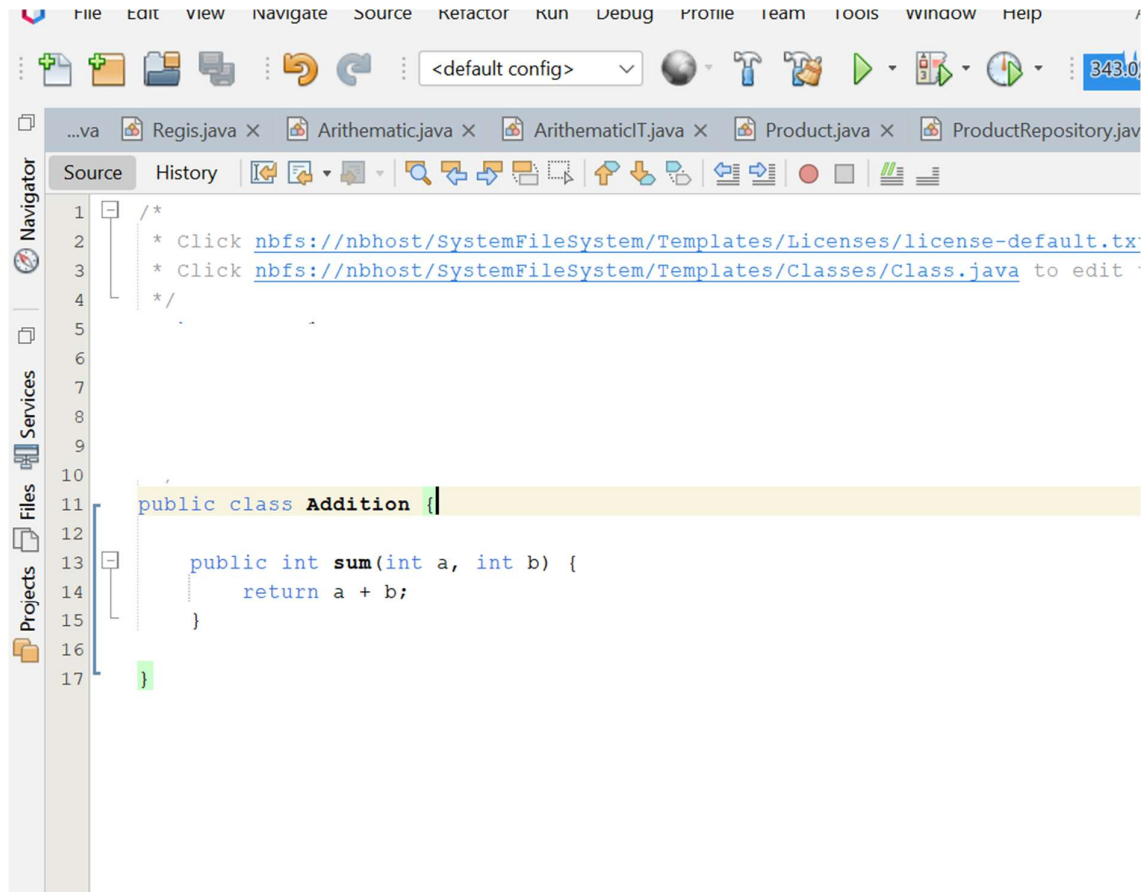
aryan1.ProductRepositoryIT x

Tests passed: 100.00 %

All 4 tests passed. (0.133 s)

Starting P
Finished P

Q14. Demonstrate test driven development through a project Addition. The project should contain a class Addition with appropriate methods for returning the sum of two numbers. Through JUnit create corresponding Parameterized Test classes and TestRunner classes to execute the above test on a collection of test cases. The test cases should be chosen using boundary value analysis technique for an input range from [1-10] for the first number and [11-20] for the second number.

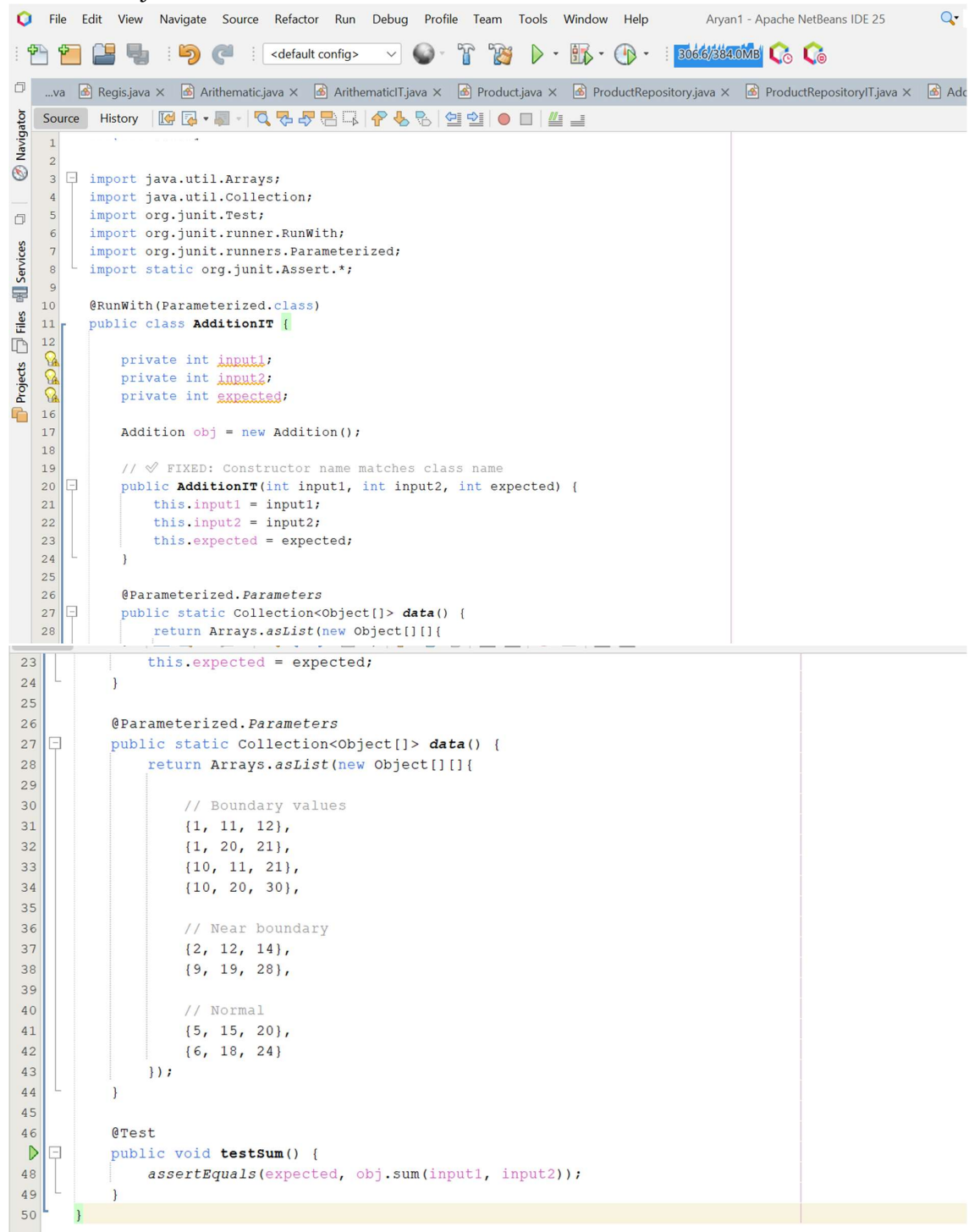


The screenshot shows an IDE window with the following components:

- Top Bar:** Menus (File, Edit, View, Navigate, Source, Refactor, Run, Debug, Profile, Team, Tools, Window, Help) and a toolbar with icons for file operations, running, and debugging. A dropdown menu shows "<default config>".
- Tab Bar:** Displays open files: "...va", "Regis.java", "Arithmetic.java", "ArithmeticIT.java", "Product.java", and "ProductRepository.jav".
- Left Sidebar:** Contains a "Navigator" pane with icons for "Source", "History", "Services", "Files", and "Projects".
- Main Editor:** Displays the source code of a Java class. The code is as follows:

```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.tx
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit
4  */
5
6
7
8
9
10
11 public class Addition {
12
13     public int sum(int a, int b) {
14         return a + b;
15     }
16
17 }
```

//AdditionIT.java



```
1
2
3 import java.util.Arrays;
4 import java.util.Collection;
5 import org.junit.Test;
6 import org.junit.runner.RunWith;
7 import org.junit.runners.Parameterized;
8 import static org.junit.Assert.*;
9
10 @RunWith(Parameterized.class)
11 public class AdditionIT {
12
13     private int input1;
14     private int input2;
15     private int expected;
16
17     Addition obj = new Addition();
18
19     // ✓ FIXED: Constructor name matches class name
20     public AdditionIT(int input1, int input2, int expected) {
21         this.input1 = input1;
22         this.input2 = input2;
23         this.expected = expected;
24     }
25
26     @Parameterized.Parameters
27     public static Collection<Object[]> data() {
28         return Arrays.asList(new Object[][]{
29
30             // Boundary values
31             {1, 11, 12},
32             {1, 20, 21},
33             {10, 11, 21},
34             {10, 20, 30},
35
36             // Near boundary
37             {2, 12, 14},
38             {9, 19, 28},
39
40             // Normal
41             {5, 15, 20},
42             {6, 18, 24}
43         });
44     }
45
46     @Test
47     public void testSum() {
48         assertEquals(expected, obj.sum(input1, input2));
49     }
50 }
```

Test Case Output:

Projects

34

35

36

37

38

39

40

41

(10, 20, 30),

// Near boundary

(2, 12, 14),

(9, 19, 28),

// Normal

(5, 15, 20)

Test Results

aryan1.ProductRepositoryIT X

aryan1.AdditionIT X

Tests passed: 100.00 %

All 8 tests passed. (0.131 s)

Q15. Demonstrate test driven development through a project MessageUtility. The project should contain a class MessageUtil with appropriate methods for printing a Message .Through JUnit create corresponding Test classes and TestRunner classes to execute and test the above class

```

1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change th
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5
6
7
8
9
10
11
12 public class MessageUtil {
13
14     private String message;
15
16     public MessageUtil(String message) {
17         this.message = message;
18     }
19
20     public String printMessage() {
21         return message;
22     }
23
24     public String updateMessage(String newMessage) {
25         this.message = newMessage;
26         return this.message;
27     }
28 }

```

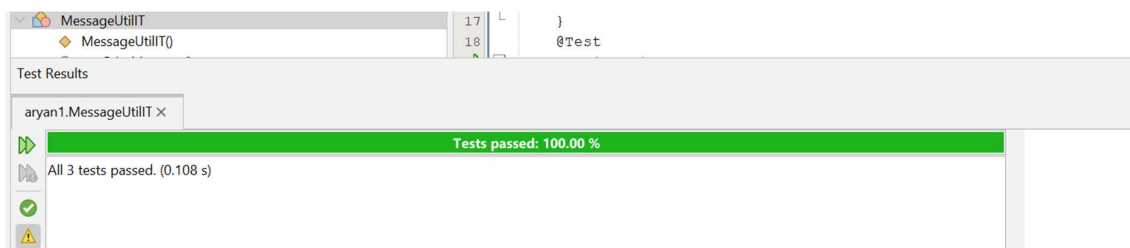
//MessageUtilIT.java

```

1
2
3 import org.junit.Test;
4 import static org.junit.Assert.*;
5
6 public class MessageUtilIT {
7     @Test
8     public void testPrintMessage() {
9         MessageUtil msg = new MessageUtil("Hello World");
10        assertEquals("Hello World", msg.printMessage());
11    }
12    @Test
13    public void testUpdateMessage() {
14        MessageUtil msg = new MessageUtil("Hello");
15        String result = msg.updateMessage("Hi");
16        assertEquals("Hi", result);
17    }
18    @Test
19    public void testPrintMessageFail() {
20        MessageUtil msg = new MessageUtil("Hello");
21        assertEquals("Hello", msg.printMessage());
22    }
23 }

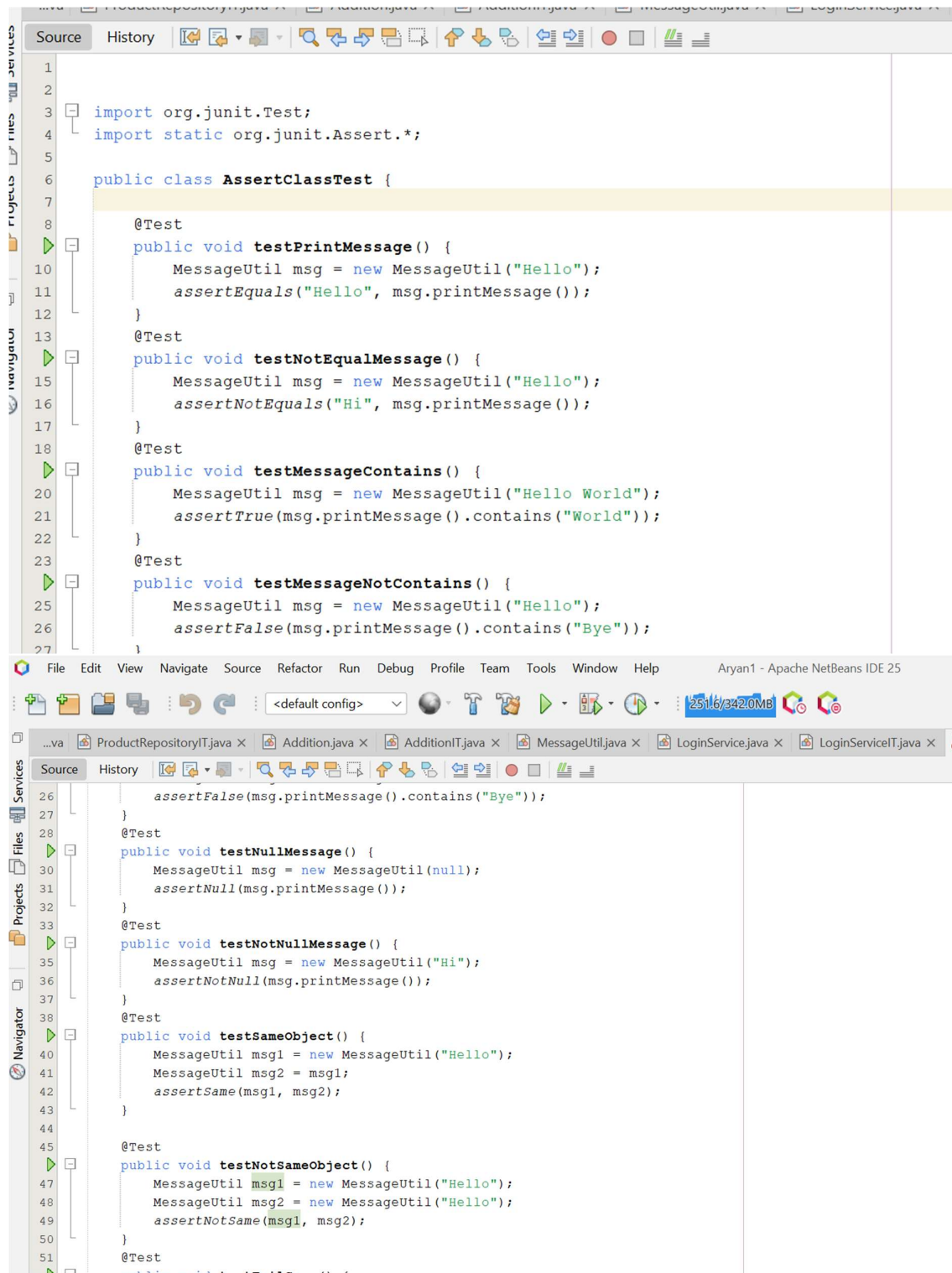
```

Test Case Output:



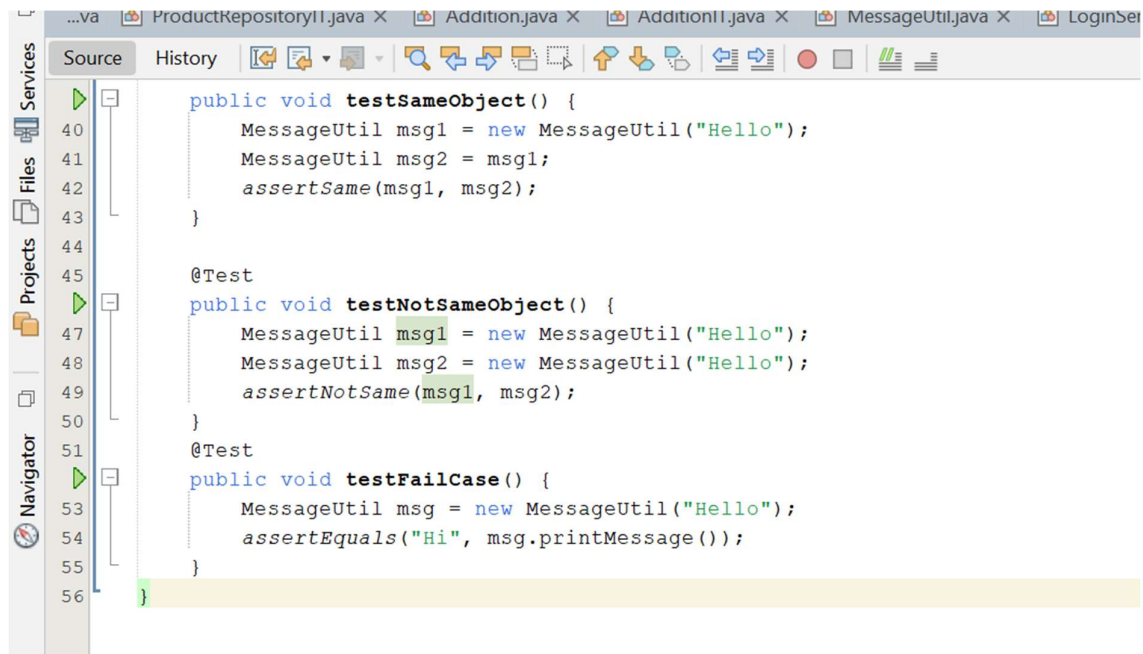
Piyush Goel (09311604425)

Q16. Create a new project called AssertClass. Refactor the test class and TestRunner class of question 15 to demonstrate the assertion methods of the assert class to write useful test.



The screenshot displays the Apache NetBeans IDE interface. The top editor shows the `AssertClassTest` class with five test methods: `testPrintMessage`, `testNotEqualMessage`, `testMessageContains`, `testMessageNotContains`, and `testNullMessage`. The bottom editor shows the continuation of the `AssertClassTest` class with three more test methods: `testNotNullMessage`, `testSameObject`, and `testNotSameObject`. The IDE's interface includes a Source tab, a History tab, a Navigator on the left, and a toolbar at the top. The status bar at the bottom indicates the file size as 2516/3420MB.

```
1
2
3 import org.junit.Test;
4 import static org.junit.Assert.*;
5
6 public class AssertClassTest {
7
8     @Test
9     public void testPrintMessage() {
10         MessageUtil msg = new MessageUtil("Hello");
11         assertEquals("Hello", msg.printMessage());
12     }
13
14     @Test
15     public void testNotEqualMessage() {
16         MessageUtil msg = new MessageUtil("Hello");
17         assertNotEquals("Hi", msg.printMessage());
18     }
19
20     @Test
21     public void testMessageContains() {
22         MessageUtil msg = new MessageUtil("Hello World");
23         assertTrue(msg.printMessage().contains("World"));
24     }
25
26     @Test
27     public void testMessageNotContains() {
28         MessageUtil msg = new MessageUtil("Hello");
29         assertFalse(msg.printMessage().contains("Bye"));
30     }
31
32     @Test
33     public void testNullMessage() {
34         MessageUtil msg = new MessageUtil(null);
35         assertNull(msg.printMessage());
36     }
37
38     @Test
39     public void testNotNullMessage() {
40         MessageUtil msg = new MessageUtil("Hi");
41         assertNotNull(msg.printMessage());
42     }
43
44     @Test
45     public void testSameObject() {
46         MessageUtil msg1 = new MessageUtil("Hello");
47         MessageUtil msg2 = msg1;
48         assertEquals(msg1, msg2);
49     }
50
51     @Test
52     public void testNotSameObject() {
53         MessageUtil msg1 = new MessageUtil("Hello");
54         MessageUtil msg2 = new MessageUtil("Hello");
55         assertNotSame(msg1, msg2);
56     }
57
58     @Test
59     public void testNotSameObject2() {
60         MessageUtil msg1 = new MessageUtil("Hello");
61         MessageUtil msg2 = new MessageUtil("Hello");
62         assertNotSame(msg1, msg2);
63     }
64 }
```



The screenshot shows an IDE with several open files: ProductRepositoryIT.java, Addition.java, AdditionIT.java, MessageUtil.java, and LoginServiceIT.java. The 'Source' tab is active, displaying the following Java code:

```
public void testSameObject() {  
    MessageUtil msg1 = new MessageUtil("Hello");  
    MessageUtil msg2 = msg1;  
    assertSame(msg1, msg2);  
}  
  
@Test  
public void testNotSameObject() {  
    MessageUtil msg1 = new MessageUtil("Hello");  
    MessageUtil msg2 = new MessageUtil("Hello");  
    assertNotSame(msg1, msg2);  
}  
  
@Test  
public void testFailCase() {  
    MessageUtil msg = new MessageUtil("Hello");  
    assertEquals("Hi", msg.printMessage());  
}
```

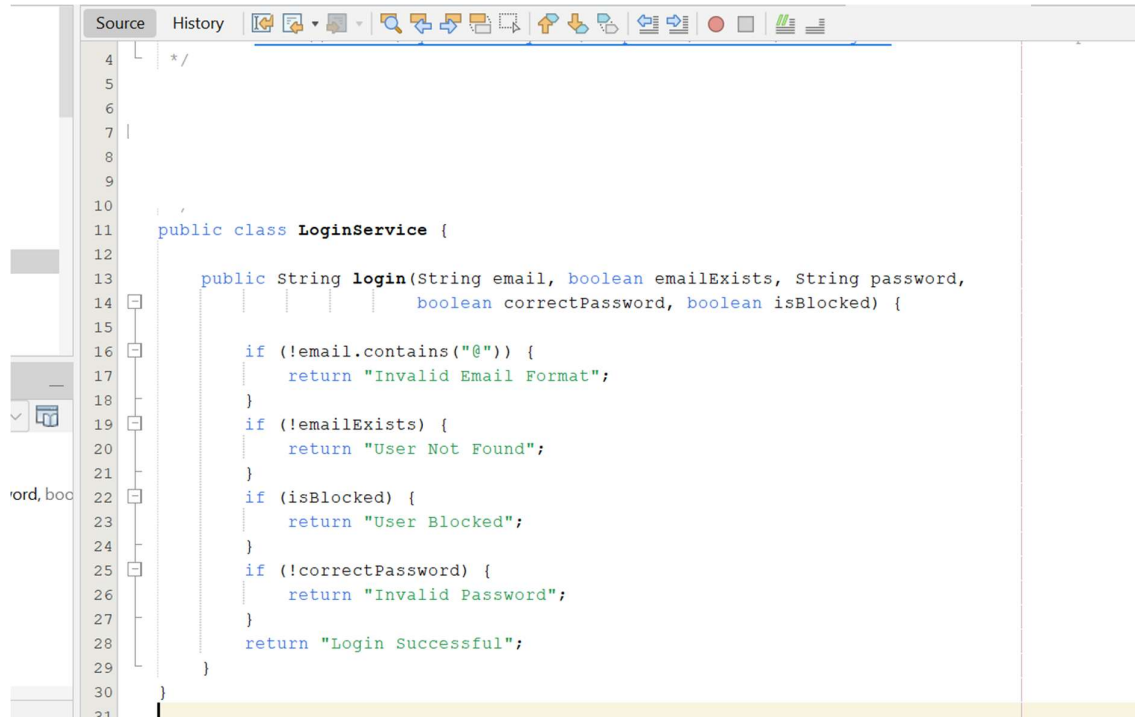
Test Case Output:



The screenshot shows the 'Test Results' window with the following details:

- Test Results window is open, showing a list of test results.
- The test results table shows 8 tests passed and 1 test failed (0.163 s).
- The failed test is `aryan1.AssertClassTest` with the message `Failed`.
- The failed test is `testFailCase` with the message `Failed: expected:<H[i]> but was:<H[ello]>`.

Q17. Create optimal parametrized tests for the login web page of a company website say XYZ Corporation. The Login feature is governed by the following specifications: • A user can enter a valid email or invalid email. • The user's email is in the database. • A user enters a correct or incorrect password (a password that doesn't fit the account or password is invalid). • A user can be blocked.

A screenshot of an IDE window showing a Java class named LoginService. The code is as follows:

```
4  */
5
6
7  |
8
9
10 /
11 public class LoginService {
12
13     public String login(String email, boolean emailExists, String password,
14                         boolean correctPassword, boolean isBlocked) {
15
16         if (!email.contains("@")) {
17             return "Invalid Email Format";
18         }
19         if (!emailExists) {
20             return "User Not Found";
21         }
22         if (isBlocked) {
23             return "User Blocked";
24         }
25         if (!correctPassword) {
26             return "Invalid Password";
27         }
28         return "Login Successful";
29     }
30 }
31
```

//LoginServiceIT.java

The image displays two screenshots of the Apache NetBeans IDE 25, showing the development of a Java class named `LoginServiceIT`.

Top Screenshot: The editor shows the class declaration and constructor. The class is annotated with `@RunWith(Parameterized.class)`. The constructor is `public LoginServiceIT(String email, boolean emailExists, String password, boolean correctPassword, boolean isBlocked, String expectedResult)`. The class has several private fields: `email`, `emailExists`, `password`, `correctPassword`, `isBlocked`, and `expectedResult`. The constructor initializes these fields using `this`.

```
1
2
3 import java.util.Arrays;
4 import java.util.Collection;
5 import org.junit.Test;
6 import org.junit.runner.RunWith;
7 import org.junit.runners.Parameterized;
8 import static org.junit.Assert.*;
9
10 @RunWith(Parameterized.class)
11 public class LoginServiceIT {
12
13     private String email;
14     private boolean emailExists;
15     private String password;
16     private boolean correctPassword;
17     private boolean isBlocked;
18     private String expectedResult;
19
20     LoginService service = new LoginService();
21
22     // Constructor name MUST match class name
23     public LoginServiceIT(String email, boolean emailExists, String password,
24                           boolean correctPassword, boolean isBlocked,
25                           String expectedResult) {
26
27         this.email = email;
28         this.emailExists = emailExists;
```

Bottom Screenshot: The editor shows the initialization of the fields and the `data()` method. The fields are initialized using `this`. The `data()` method returns a collection of test data.

```
29         this.password = password;
30         this.correctPassword = correctPassword;
31         this.isBlocked = isBlocked;
32         this.expectedResult = expectedResult;
33     }
34
35     @Parameterized.Parameters
36     public static Collection<Object[]> data() {
37         return Arrays.asList(new Object[][]{
38
39             // X Invalid email format
40             {"invalidEmail", false, "1234", false, false, "Invalid Email Format"},
41
42             // X Email not in DB
43             {"user@xyz.com", false, "1234", false, false, "User Not Found"},
44
45             // X Incorrect password
46             {"user@xyz.com", true, "wrong", false, false, "Invalid Password"},
47
48             // X Blocked user
49             {"blocked@xyz.com", true, "1234", true, true, "User Blocked"},
50
51             // X Successful login
52             {"user@xyz.com", true, "1234", true, false, "Login Successful"}
53         });
54     }
55 }
```

```
46         { "user@xyz.com", true, wrong, false, false, invalidPassword },
47
48         // ✖ Blocked user
49         { "blocked@xyz.com", true, "1234", true, true, "User Blocked"},
50
51         // ✔ Successful login
52         { "user@xyz.com", true, "1234", true, false, "Login Successful"}
53     });
54 }
55
56 @Test
57 public void testLogin() {
58     String result = service.login(email, emailExists, password, correctPassword, isBlocked);
59     assertEquals(expectedResult, result);
60 }
61
```

Test Case Output:

```
49         { "blocked@xyz.com", true, "1234", true, true, "User Blocked"},
50
51         // ✔ Successful login
52         { "user@xyz.com", true, "1234", true, false, "Login Successful"}
53     });
54 }
55
56 @Test
57 public void testLogin() {
58     String result = service.login(email, emailExists, password, correctPassword, isBlocked);
59     assertEquals(expectedResult, result);
60 }
```

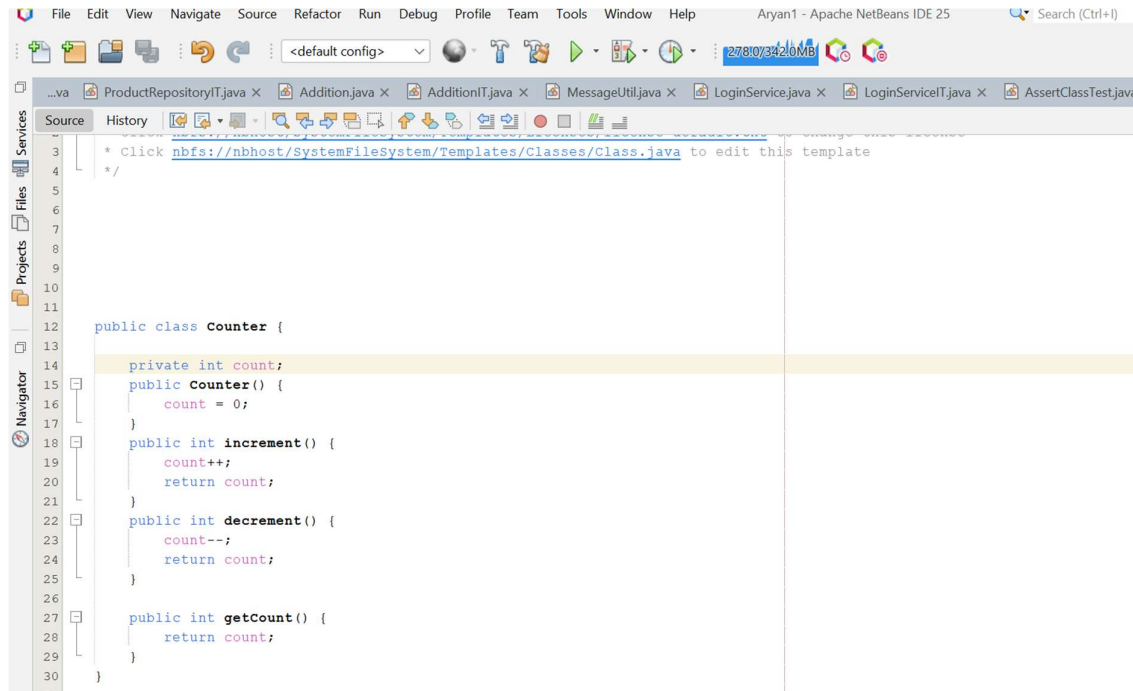
Test Results ×

aryan1.LoginServiceIT ×

Tests passed: 100.00 %

All 5 tests passed. (0.13 s)

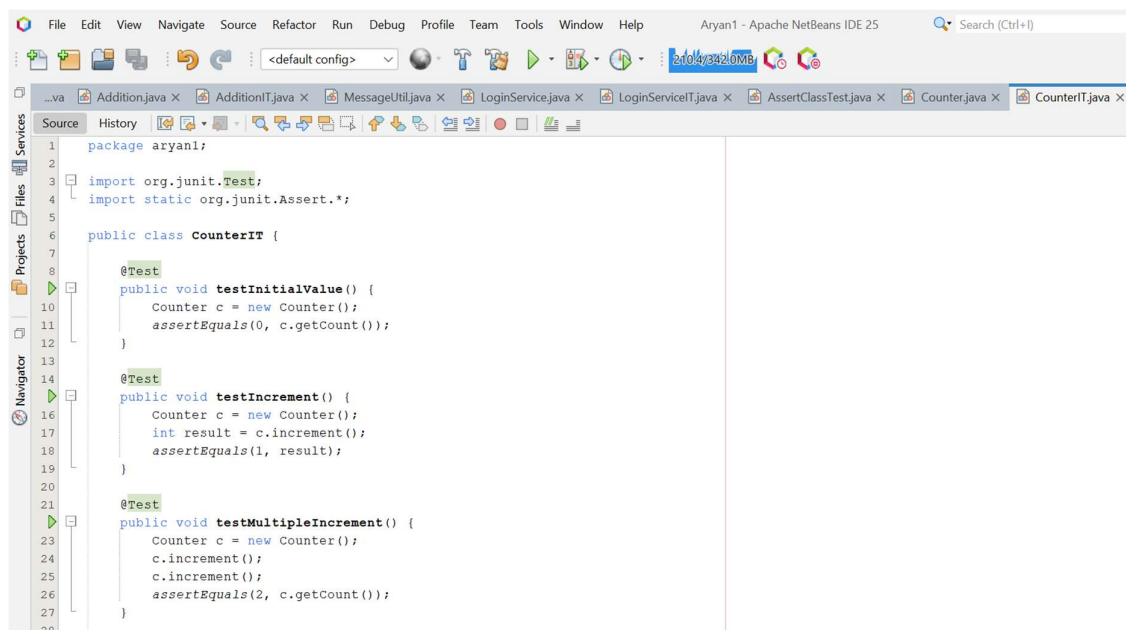
Q18. Create a trivial class Counter with the following: -The constructor will create a counter and set it to 0 -The increment method will add one to the counter and return the new value. - The decrement method will subtract one from the counter and return the new value. Write test methods to test these methods. Also create a TestRunner class to execute these methods.



The screenshot shows the Apache NetBeans IDE 25 interface. The main editor window displays the code for the `Counter` class. The code includes a private integer `count`, a constructor `Counter()` that initializes `count` to 0, and three public methods: `increment()`, `decrement()`, and `getCount()`. The `increment()` method increments `count` and returns the new value. The `decrement()` method decrements `count` and returns the new value. The `getCount()` method returns the current value of `count`. The IDE's Project and Files views are visible on the left, and the Run toolbar is at the top.

```
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5
6
7
8
9
10
11
12 public class Counter {
13
14     private int count;
15     public Counter() {
16         count = 0;
17     }
18     public int increment() {
19         count++;
20         return count;
21     }
22     public int decrement() {
23         count--;
24         return count;
25     }
26
27     public int getCount() {
28         return count;
29     }
30 }
31
```

//CounterIT.java



The screenshot shows the Apache NetBeans IDE 25 interface with the `CounterIT.java` file open. The code defines the `CounterIT` class, which contains three JUnit test methods: `testInitialValue()`, `testIncrement()`, and `testMultipleIncrement()`. Each test method creates a new `Counter` object and performs assertions to verify its behavior. The IDE's Project and Files views are visible on the left, and the Run toolbar is at the top.

```
1 package aryan1;
2
3 import org.junit.Test;
4 import static org.junit.Assert.*;
5
6 public class CounterIT {
7
8     @Test
9     public void testInitialValue() {
10         Counter c = new Counter();
11         assertEquals(0, c.getCount());
12     }
13
14     @Test
15     public void testIncrement() {
16         Counter c = new Counter();
17         int result = c.increment();
18         assertEquals(1, result);
19     }
20
21     @Test
22     public void testMultipleIncrement() {
23         Counter c = new Counter();
24         c.increment();
25         c.increment();
26         assertEquals(2, c.getCount());
27     }
28 }

```

```
23     Counter c = new Counter();
24     c.increment();
25     c.increment();
26     assertEquals(2, c.getCount());
27 }
28
29 @Test
30 public void testDecrement() {
31     Counter c = new Counter();
32     int result = c.decrement();
33     assertEquals(-1, result);
34 }
35
36 @Test
37 public void testFailCase() {
38     Counter c = new Counter();
39     c.increment();
40     assertEquals(2, c.getCount());
```

Test Case Output:

```
25     c.increment();
26     assertEquals(2, c.getCount());
27 }
28
29 @Test
30 public void testDecrement() {
31     Counter c = new Counter();
32     int result = c.decrement();
33     assertEquals(-1, result);
34 }
35
36 @Test
37 public void testFailCase() {
38     Counter c = new Counter();
```

Test Results

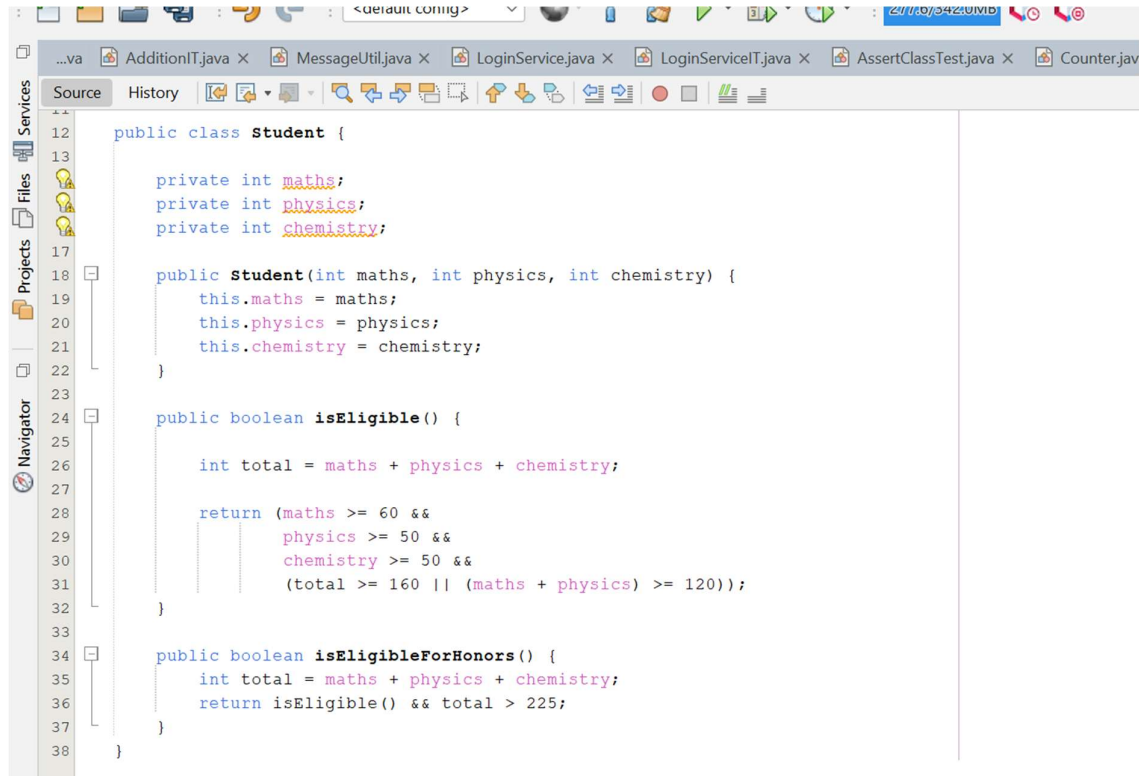
aryan1.CounterIT ×

Tests passed: 80.00 %

4 tests passed, 1 test failed. (0.154 s)

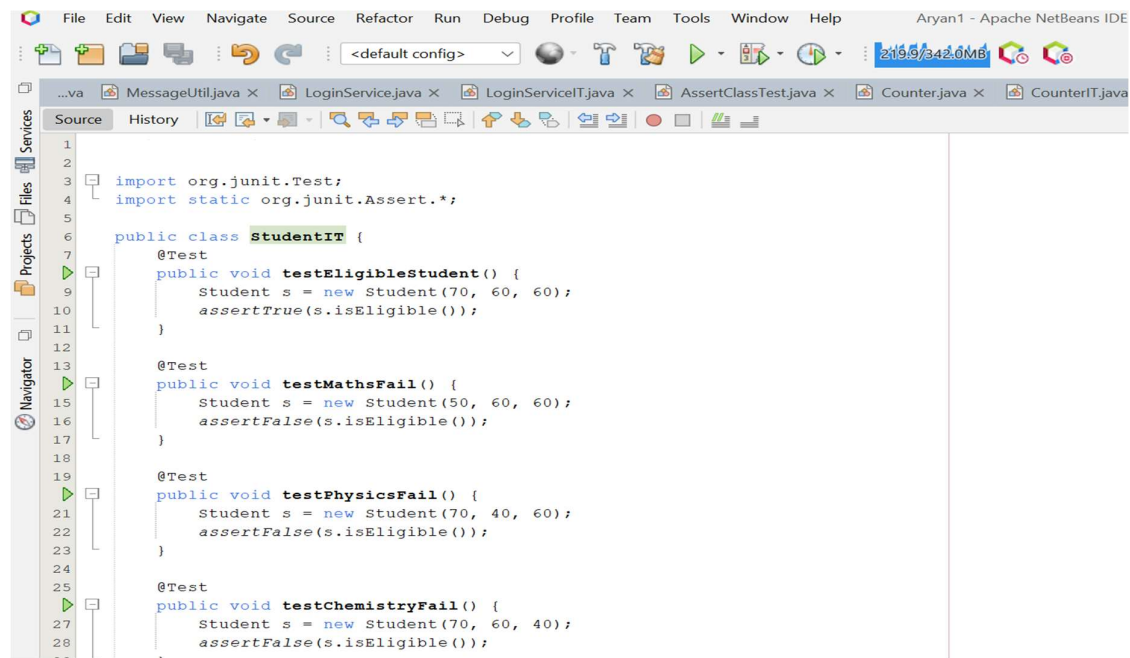
- aryan1.CounterIT Failed
 - testFailCase Failed: expected:<2> but was:<1>

Q19. Demonstrate test driven development through a project Project Repository. The project should contain based on the following rules: a) Marks in Mathematics $\geq$ 60 b) Marks in Physics $\geq$ 50 c) Marks in Chemistry $\geq$ 50 d) Total in all three subjects $\geq$ 160 or e) Total in Maths & Physics $\geq$ 120 If aggregate of an eligible candidate is greater than 225, he will be eligible for honors course. Write appropriate class to represent this situation. Through JUnit create corresponding Test classes for each class. Aggregate these different tests in a test suite and execute the same

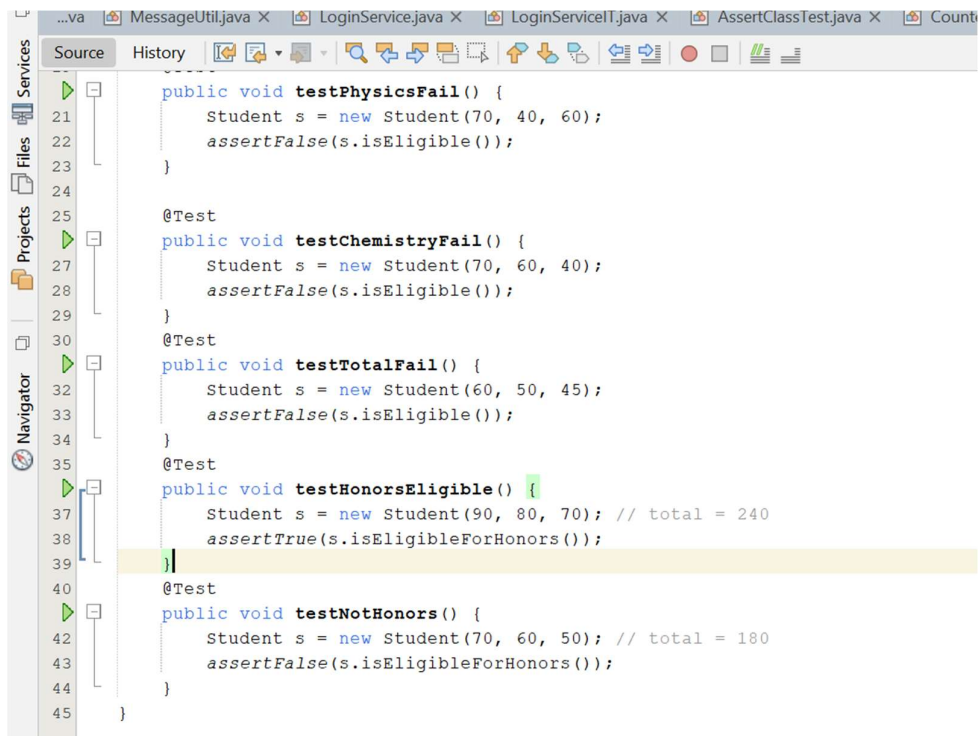
The image is a screenshot of an IDE window showing a Java class named 'Student'. The window has a title bar with several open files: 'AdditionIT.java', 'MessageUtil.java', 'LoginService.java', 'LoginServiceIT.java', 'AssertClassTest.java', and 'Counter.jav'. The 'Source' tab is active, displaying the code for the 'Student' class. The code includes private fields for 'maths', 'physics', and 'chemistry', a constructor to initialize them, and two public methods: 'isEligible()' which checks the individual subject marks and the total marks, and 'isEligibleForHonors()' which checks the total marks against 225. The IDE interface includes a 'Navigator' on the left and a toolbar at the top.

```
12 public class Student {
13
14     private int maths;
15     private int physics;
16     private int chemistry;
17
18     public Student(int maths, int physics, int chemistry) {
19         this.maths = maths;
20         this.physics = physics;
21         this.chemistry = chemistry;
22     }
23
24     public boolean isEligible() {
25
26         int total = maths + physics + chemistry;
27
28         return (maths >= 60 &&
29                 physics >= 50 &&
30                 chemistry >= 50 &&
31                 (total >= 160 || (maths + physics) >= 120));
32     }
33
34     public boolean isEligibleForHonors() {
35         int total = maths + physics + chemistry;
36         return isEligible() && total > 225;
37     }
38 }
```

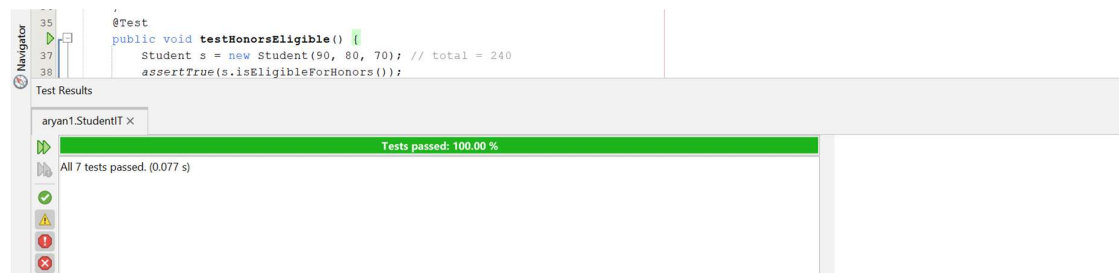
//StudentIT.java



```
1
2
3 import org.junit.Test;
4 import static org.junit.Assert.*;
5
6 public class StudentIT {
7     @Test
8     public void testEligibleStudent() {
9         Student s = new Student(70, 60, 60);
10        assertTrue(s.isEligible());
11    }
12
13    @Test
14    public void testMathsFail() {
15        Student s = new Student(50, 60, 60);
16        assertFalse(s.isEligible());
17    }
18
19    @Test
20    public void testPhysicsFail() {
21        Student s = new Student(70, 40, 60);
22        assertFalse(s.isEligible());
23    }
24
25    @Test
26    public void testChemistryFail() {
27        Student s = new Student(70, 60, 40);
28        assertFalse(s.isEligible());
29    }
30}
```



```
21 public void testPhysicsFail() {
22     Student s = new Student(70, 40, 60);
23     assertFalse(s.isEligible());
24 }
25
26 @Test
27 public void testChemistryFail() {
28     Student s = new Student(70, 60, 40);
29     assertFalse(s.isEligible());
30 }
31
32 @Test
33 public void testTotalFail() {
34     Student s = new Student(60, 50, 45);
35     assertFalse(s.isEligible());
36 }
37
38 @Test
39 public void testHonorsEligible() {
40     Student s = new Student(90, 80, 70); // total = 240
41     assertTrue(s.isEligibleForHonors());
42 }
43
44 @Test
45 public void testNotHonors() {
46     Student s = new Student(70, 60, 50); // total = 180
47     assertFalse(s.isEligibleForHonors());
48 }
49 }
```



Test Name	Status	Duration
aryan1.StudentIT	Passed	0.077 s

Test Results

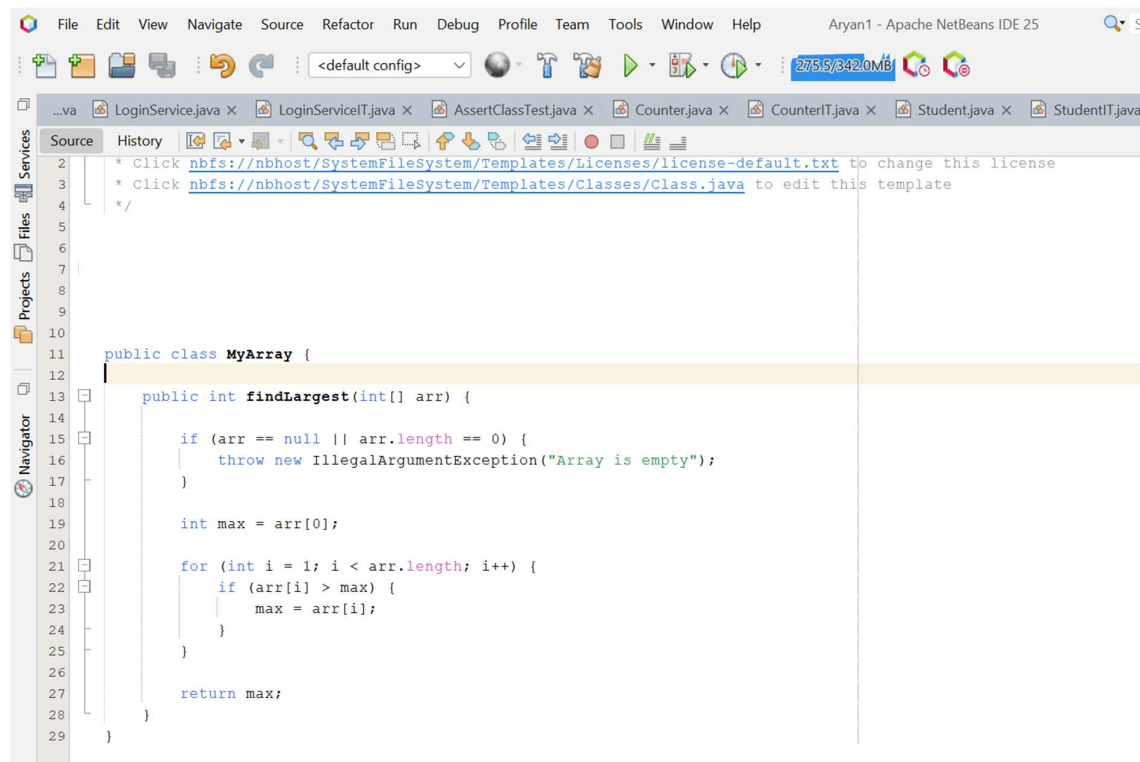
aryan1.StudentIT

Tests passed: 100.00 %

All 7 tests passed. (0.077 s)

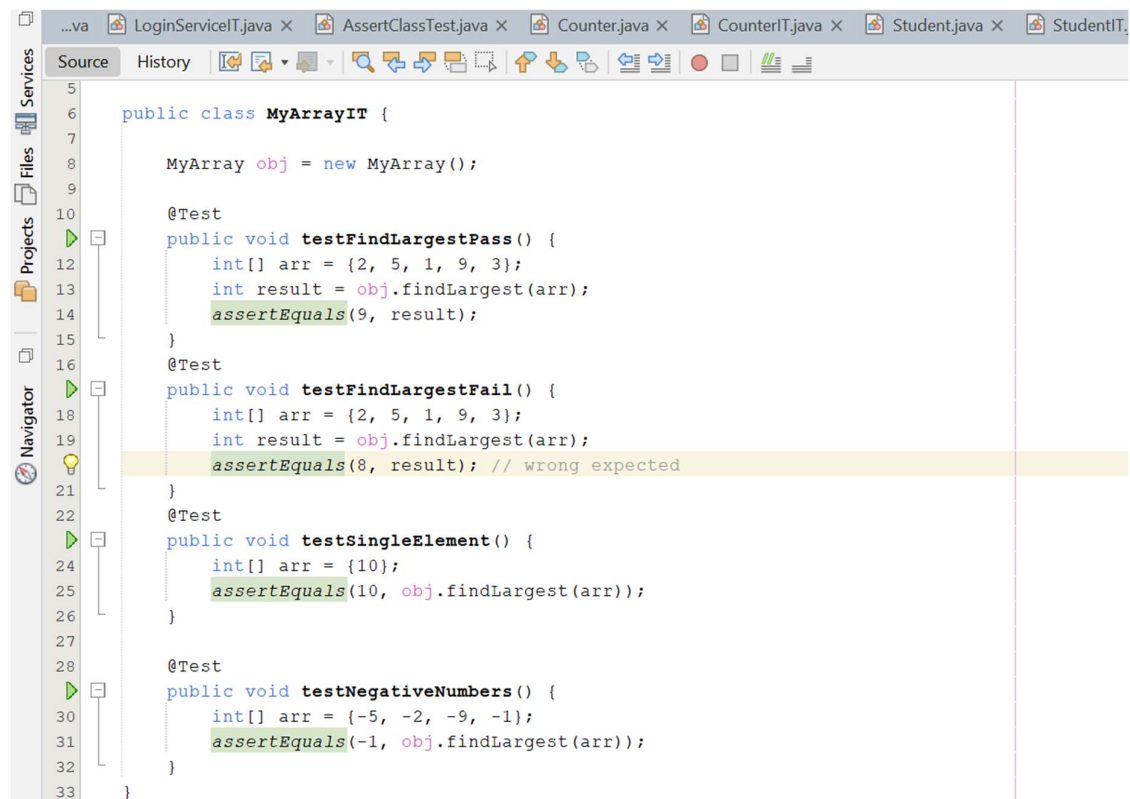
Piyush Goel (09311604425)

Q20. Demonstrate test driven development through a class MyArray. Write a linear 25- 05- search method to find out the largest element of the array. Design a test class to 2023 test this method, using JUNIT framework. Record both test cases in proper test case templates.



```
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help Aryan1 - Apache NetBeans IDE 25
...va LoginService.java X LoginServiceIT.java X AssertClassTest.java X Counter.java X CounterIT.java X Student.java X StudentIT.java
Source History
2 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4 */
5
6
7
8
9
10
11 public class MyArray {
12
13     public int findLargest(int[] arr) {
14
15         if (arr == null || arr.length == 0) {
16             throw new IllegalArgumentException("Array is empty");
17         }
18
19         int max = arr[0];
20
21         for (int i = 1; i < arr.length; i++) {
22             if (arr[i] > max) {
23                 max = arr[i];
24             }
25         }
26
27         return max;
28     }
29 }
```

//MyArrayIT.java



```
...va LoginServiceIT.java X AssertClassTest.java X Counter.java X CounterIT.java X Student.java X StudentIT.java
Source History
5
6 public class MyArrayIT {
7
8     MyArray obj = new MyArray();
9
10     @Test
11     public void testFindLargestPass() {
12         int[] arr = {2, 5, 1, 9, 3};
13         int result = obj.findLargest(arr);
14         assertEquals(9, result);
15     }
16
17     @Test
18     public void testFindLargestFail() {
19         int[] arr = {2, 5, 1, 9, 3};
20         int result = obj.findLargest(arr);
21         assertEquals(8, result); // wrong expected
22     }
23
24     @Test
25     public void testSingleElement() {
26         int[] arr = {10};
27         assertEquals(10, obj.findLargest(arr));
28     }
29
30     @Test
31     public void testNegativeNumbers() {
32         int[] arr = {-5, -2, -9, -1};
33         assertEquals(-1, obj.findLargest(arr));
34     }
35 }
```

Piyush Goel (09311604425)

Test Case Ouput:

Test Results

aryan1.StudentIT × aryan1.MyArrayIT ×

Tests passed: 75.00 %

3 tests passed, 1 test failed. (0.083 s)

aryan1.MyArrayIT Failed

testFindLargestFail Failed: expected:<8> but was:<9>